



Algorithme de Ford-Johnson



Sommaire

- Introduction
- Explication théorique de l'algorithme
 - Merge
 - Recherche dichotomique
 - Insertion
- Traduire l'algorithme en code
- Conseils / Rappels

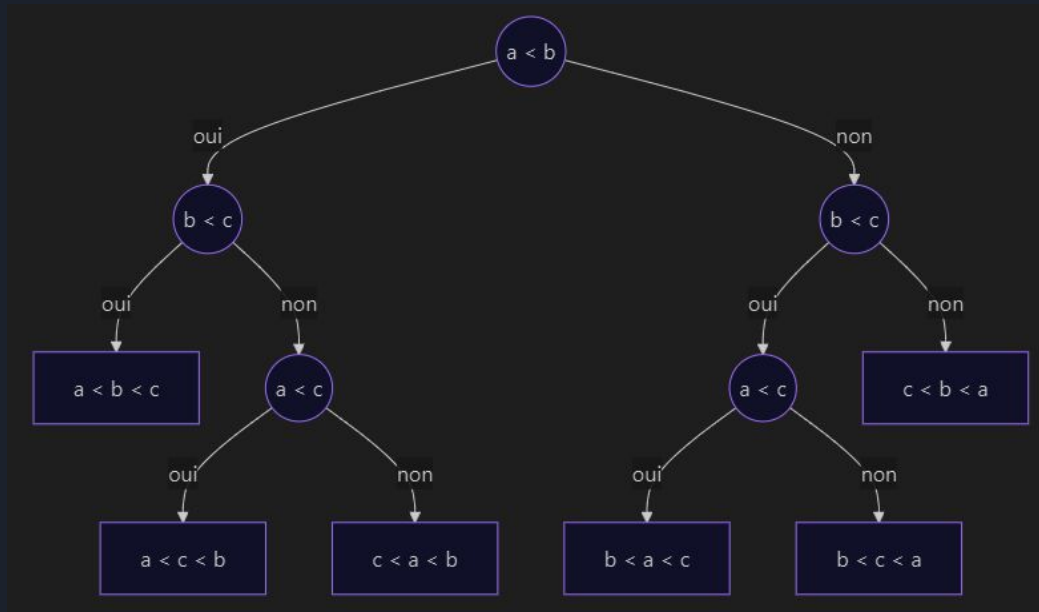
Introduction



Objectif de l'algorithme

L'algorithme de Ford-Johnson est un algorithme de **tri** qui cherche à **minimiser le nombre de comparaisons** dans le **pire des cas**.

Arbre de comparaisons

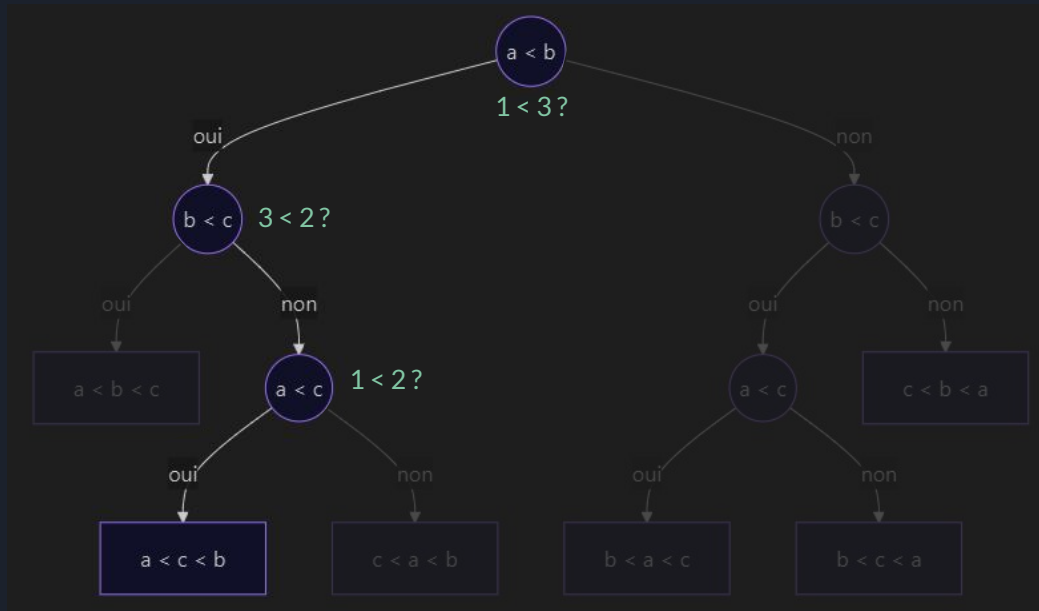


Arbre de comparaisons

$a = 1$

$b = 3$

$c = 2$

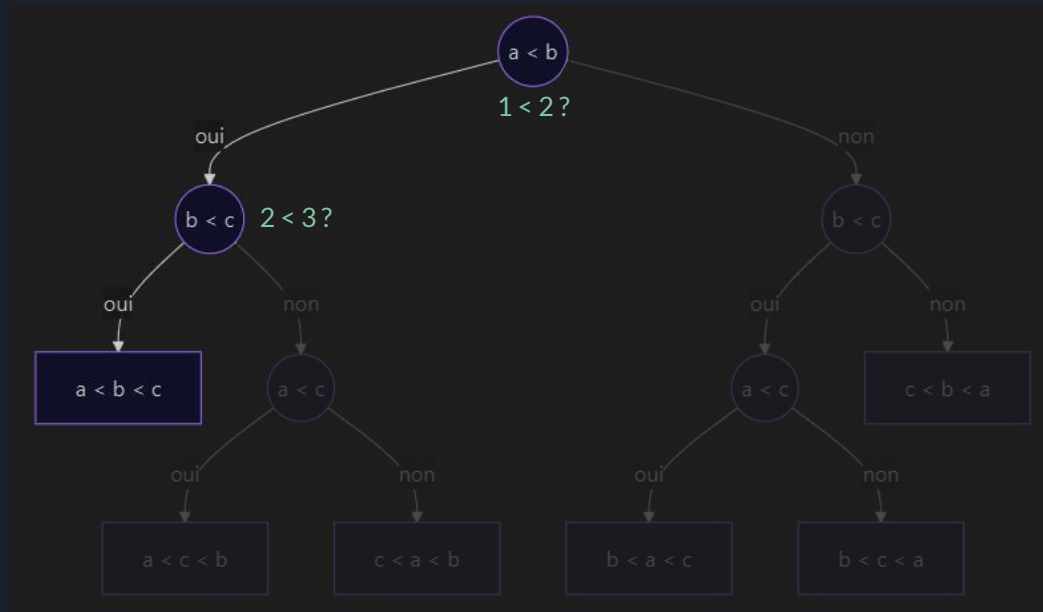


Arbre de comparaisons

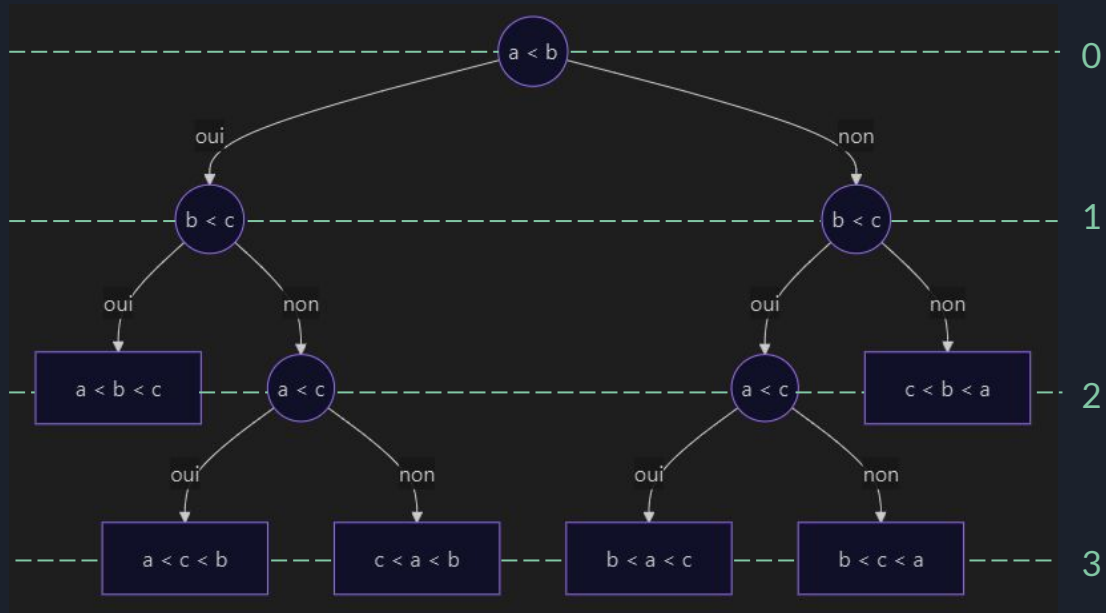
$a = 1$

$b = 2$

$c = 3$



Arbre de comparaisons





Pour aller plus loin

Nombre d'éléments à trier	Nombre de comparaisons (dans le pire des cas)
≤ 22	minimum de comparaisons possibles
≤ 46	algorithme avec le moins de comparaisons actuellement



Doublons: Fonctionnement identique, avec le même nombre de comparaisons.

Autre nom: Tri par fusion-insertion / merge-insertion sort.



Étapes

Merge

- 1) Former des paires d'éléments
(si le nombre d'éléments est impair, le dernier élément sera dans une "paire fictive")
- 2) Trier les 2 éléments de chaque paire pour former un "perdant" et un "gagnant"
(si le nombre d'éléments est impair, l'élément restant est considéré comme le perdant de sa "paire fictive")



Étapes

- 1) Former des paires d'éléments
(si le nombre d'éléments est impair, le dernier élément sera dans une "paire fictive")
- 2) Trier les 2 éléments de chaque paire pour former un "perdant" et un "gagnant"
(si le nombre d'éléments est impair, l'élément restant est considéré comme le perdant de sa "paire fictive")
- 3) Trier récursivement les paires à partir des gagnants de chaque paire

Appel récursif
de la fonction





Étapes

- 1) Former des paires d'éléments
(si le nombre d'éléments est impair, le dernier élément sera dans une "paire fictive")
- 2) Trier les 2 éléments de chaque paire pour former un "perdant" et un "gagnant"
(si le nombre d'éléments est impair, l'élément restant est considéré comme le perdant de sa "paire fictive")
- 3) Trier récursivement les paires à partir des gagnants de chaque paire
- 4) Séparer les perdants et les gagnants
- 5) Insérer le perdant de la première paire dans les gagnants
- 6) Insérer, grâce à la recherche dichotomique, les autres perdants en utilisant la suite de Jacobsthal pour sélectionner l'ordre d'insertion des perdants

Insertion



Pseudo-code

S'il y a moins de 2 éléments

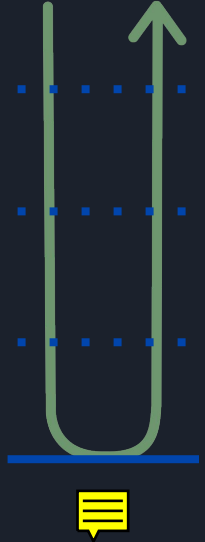
Arrêt de la récursion // Car pas assez d'éléments pour former une paire.

Merge

Appel récursif

S'il y a au moins 3 éléments // Sinon les éléments sont déjà triés, donc pas d'insertion.

Insertion





Notation

Mathématiques :

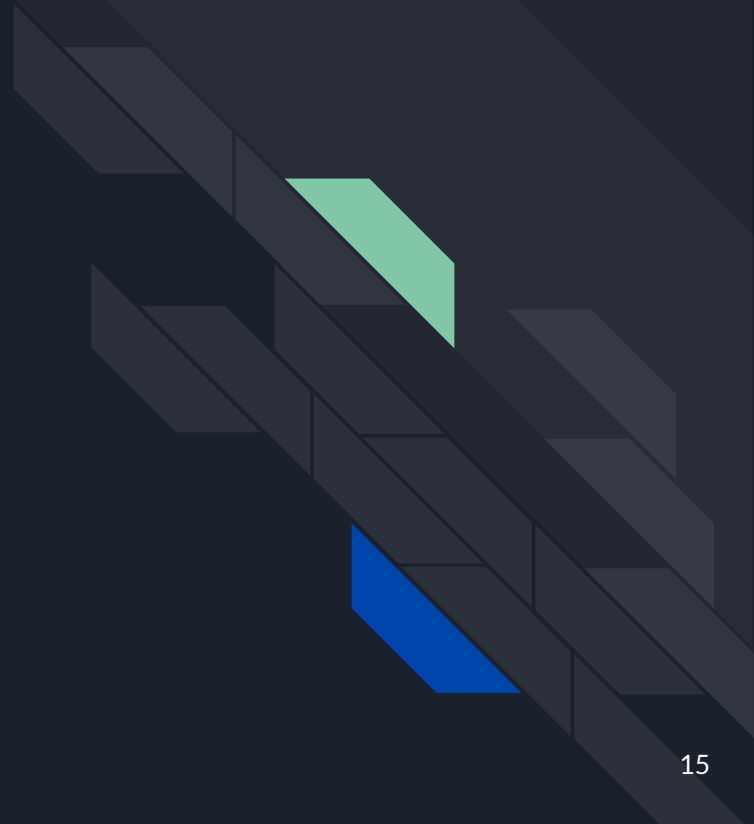
$\lfloor x \rfloor$: valeur arrondie par défaut (ex: $\lfloor 9/10 \rfloor = \lfloor 0.9 \rfloor = 0$).

$\lceil x \rceil$: valeur arrondie par excès (ex: $\lceil 1/10 \rceil = \lceil 0.1 \rceil = 1$).

Présentation :

n : nombre d'éléments.

Merge





Former des paires d'éléments

étape de
réursion: 0

10	8	7	9	6	3	4	5	2	1	11
----	---	---	---	---	---	---	---	---	---	----

Former des paires d'éléments

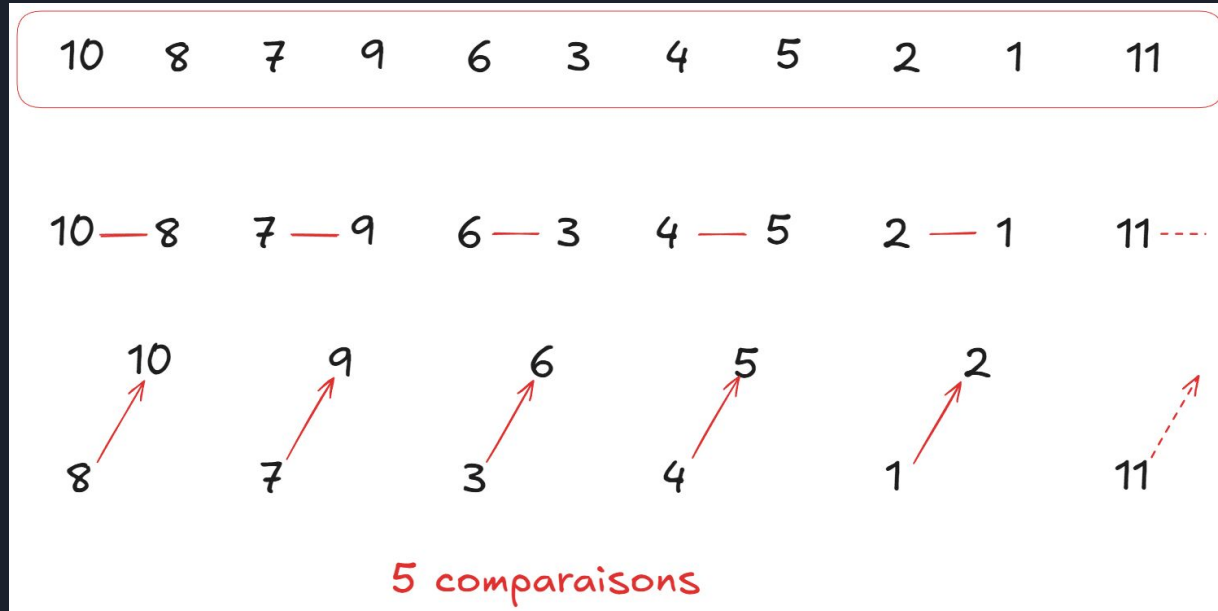
étape de
réursion: 0

10 8 7 9 6 3 4 5 2 1 11

10 — 8 7 — 9 6 — 3 4 — 5 2 — 1 11 — ...

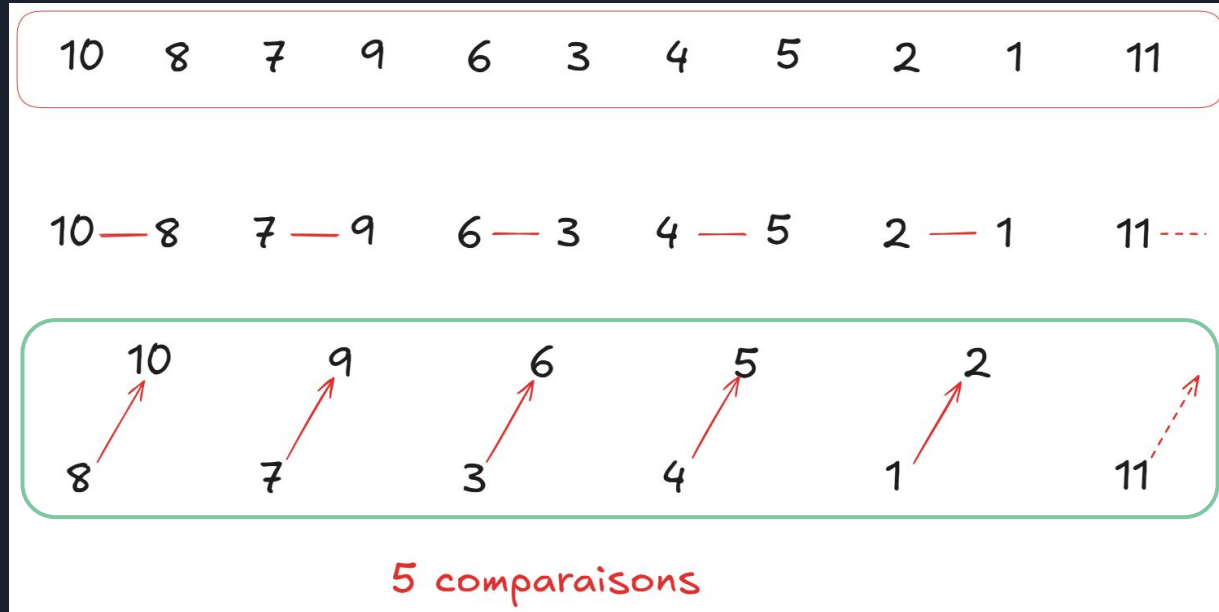
Trier les 2 éléments de chaque paire

étape de
réursion: 0



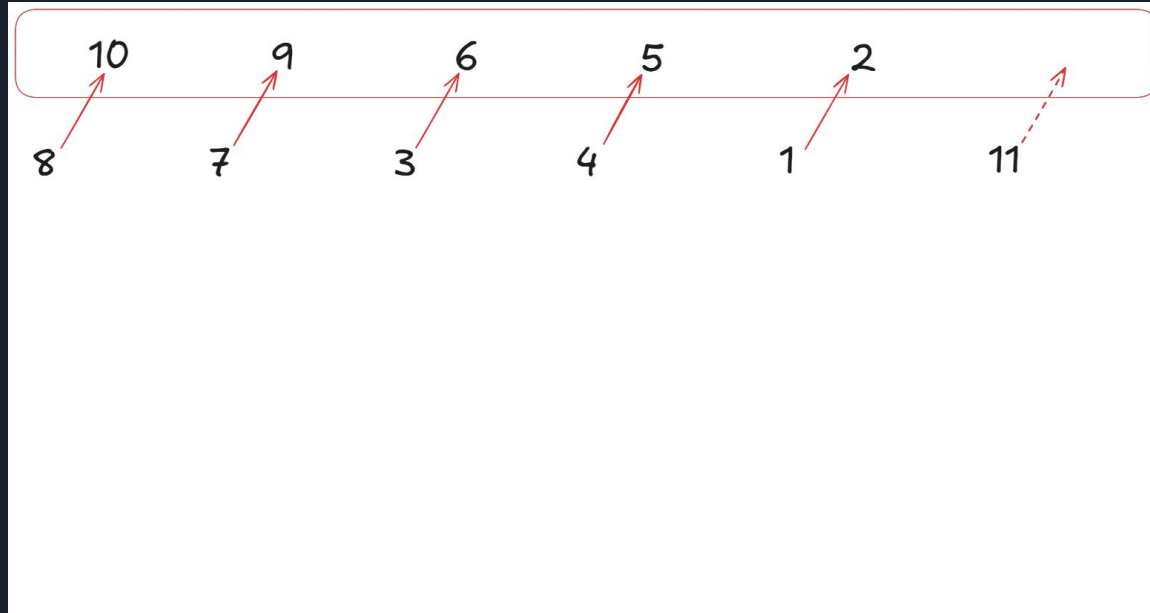
Trier récursivement à partir des gagnants

étape de
réursion: 0



Trier récursivement à partir des gagnants

étape de
réursion: 1



Trier récursivement à partir des gagnants

étape de
réursion: 1

10

9

6

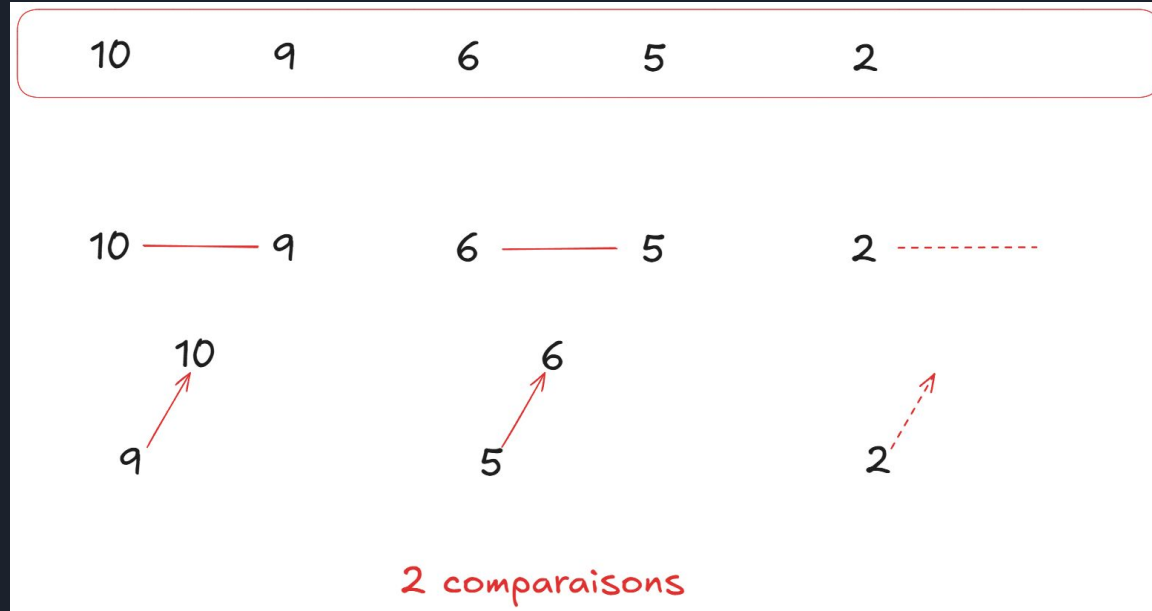
5

2



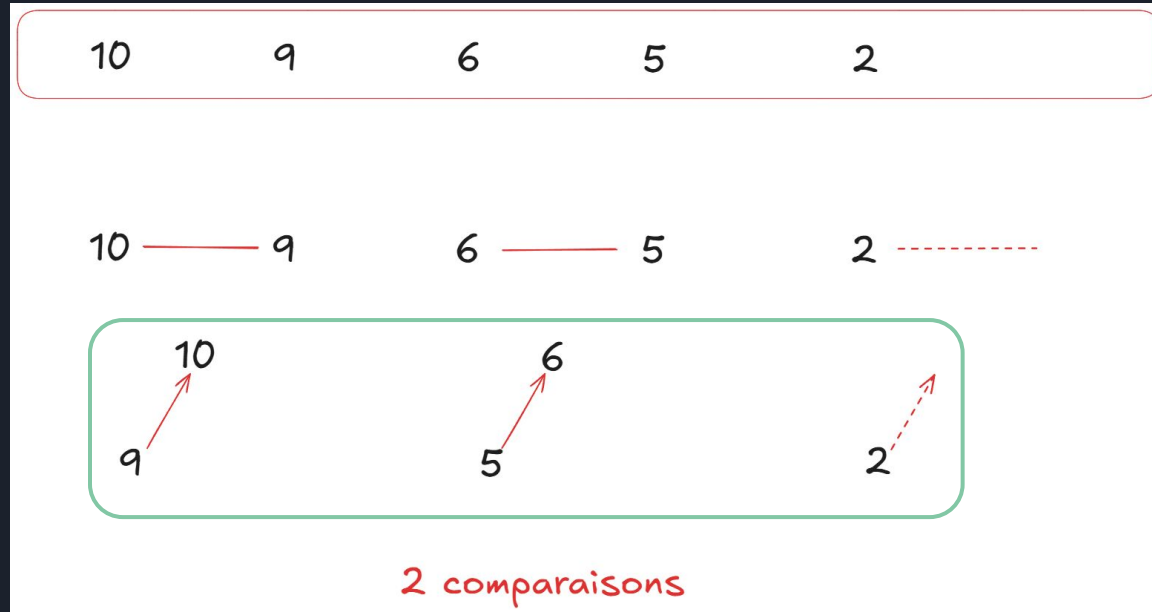
Trier récursivement à partir des gagnants

étape de
réursion: 1



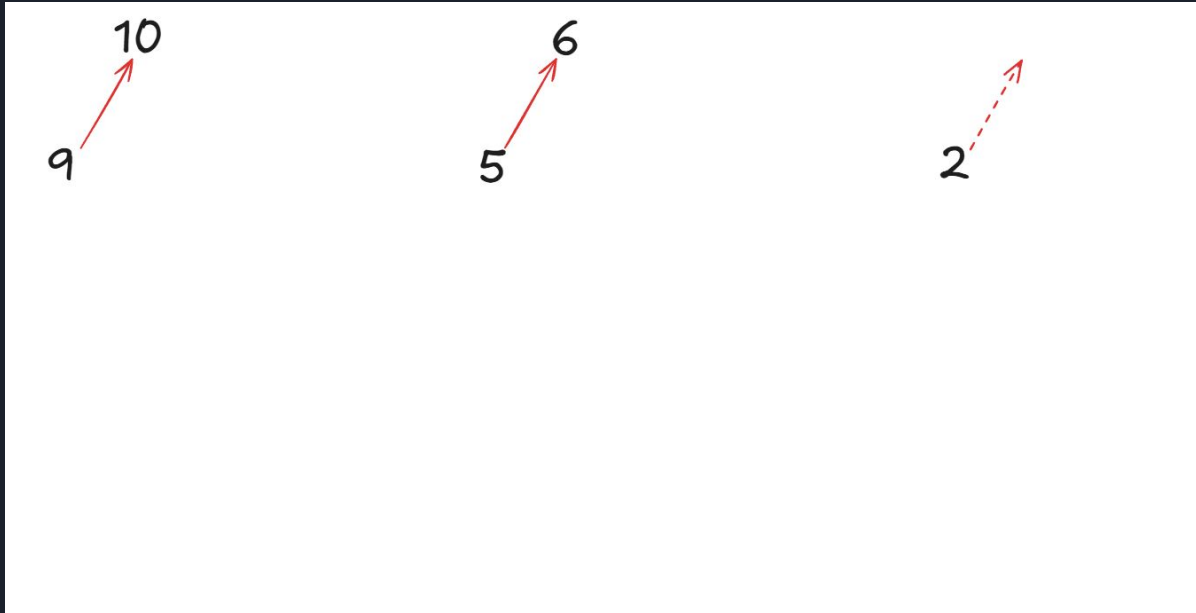
Trier récursivement à partir des gagnants

étape de
réursion: 1



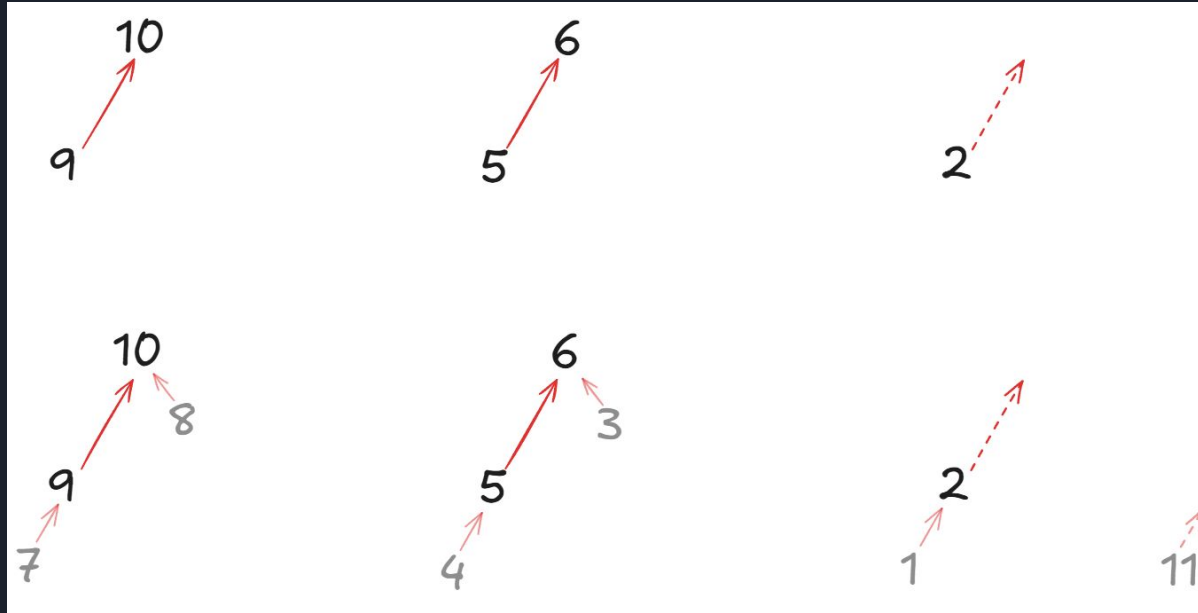
Trier récursivement à partir des gagnants

étape de
réursion: 2



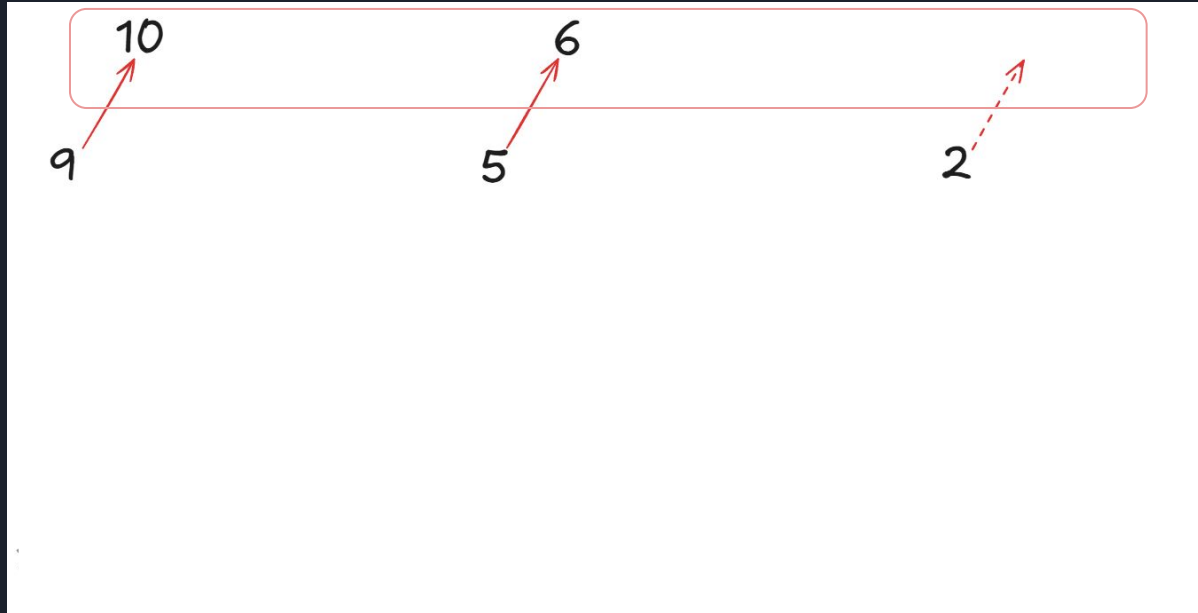
Trier récursivement à partir des gagnants

étape de
réursion: 2



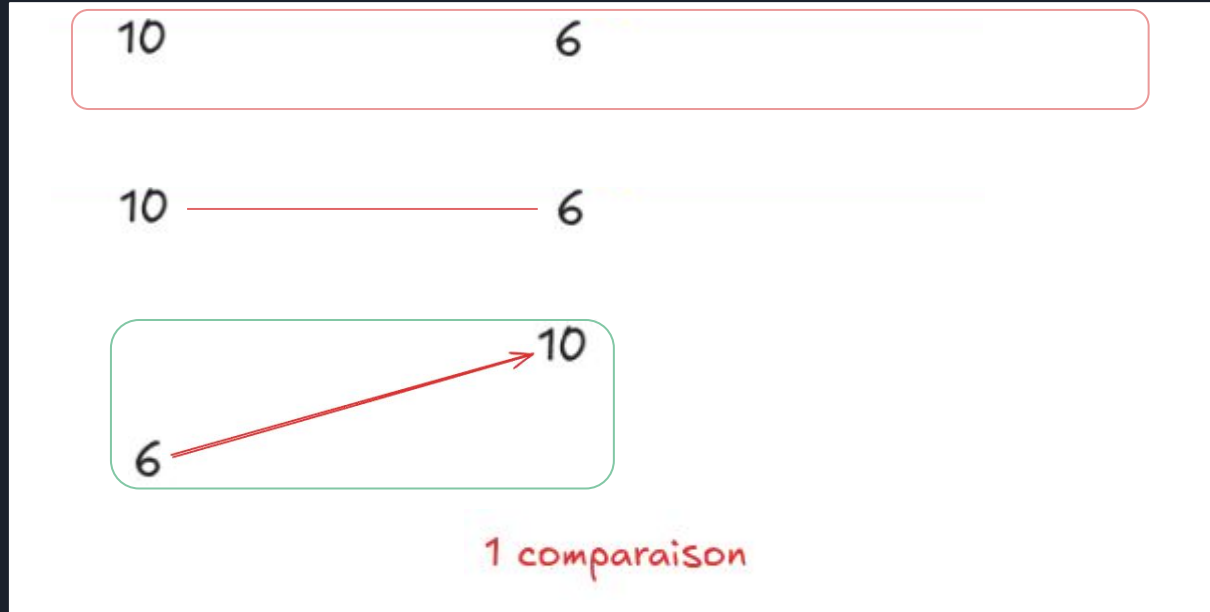
Trier récursivement à partir des gagnants

étape de
réursion: 2



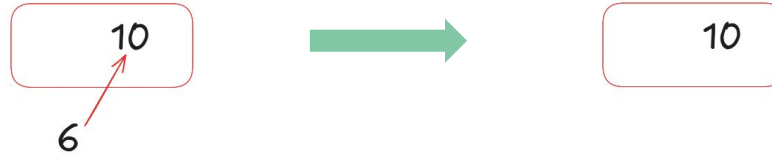
Trier récursivement à partir des gagnants

étape de
réursion : 2



Trier récursivement à partir des gagnants

étape de
réursion: 3



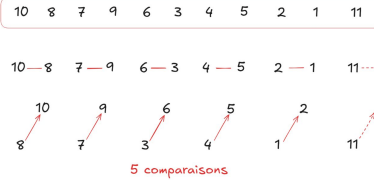
$n < 2?$

→ Condition d'arrêt de la récursion

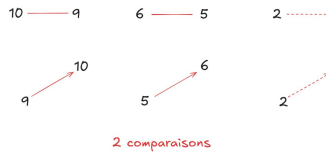
Nombre de comparaisons pour le merge

étape de récursion

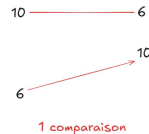
0



1



2



3

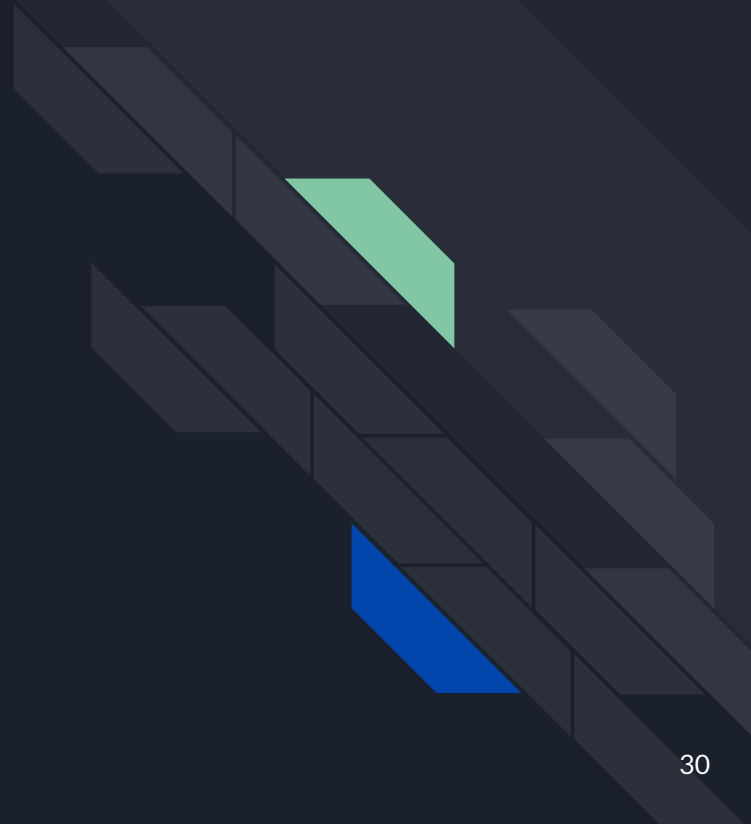
10—...
Fin de récursion

$$5 + 2 + 1 = 8 \text{ comparaisons}$$

Soit $\lfloor \ln / 2 \rfloor$ par étape



Recherche dichotomique





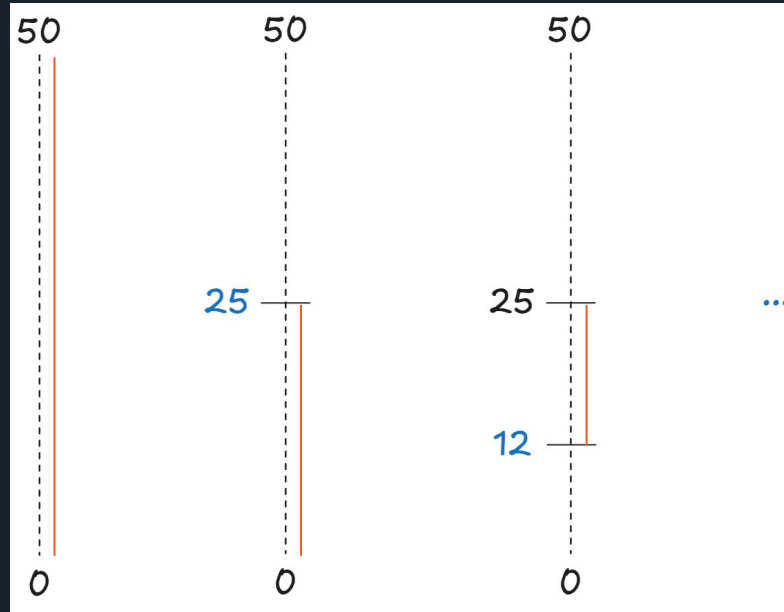
Définition

En anglais, `binary search`.

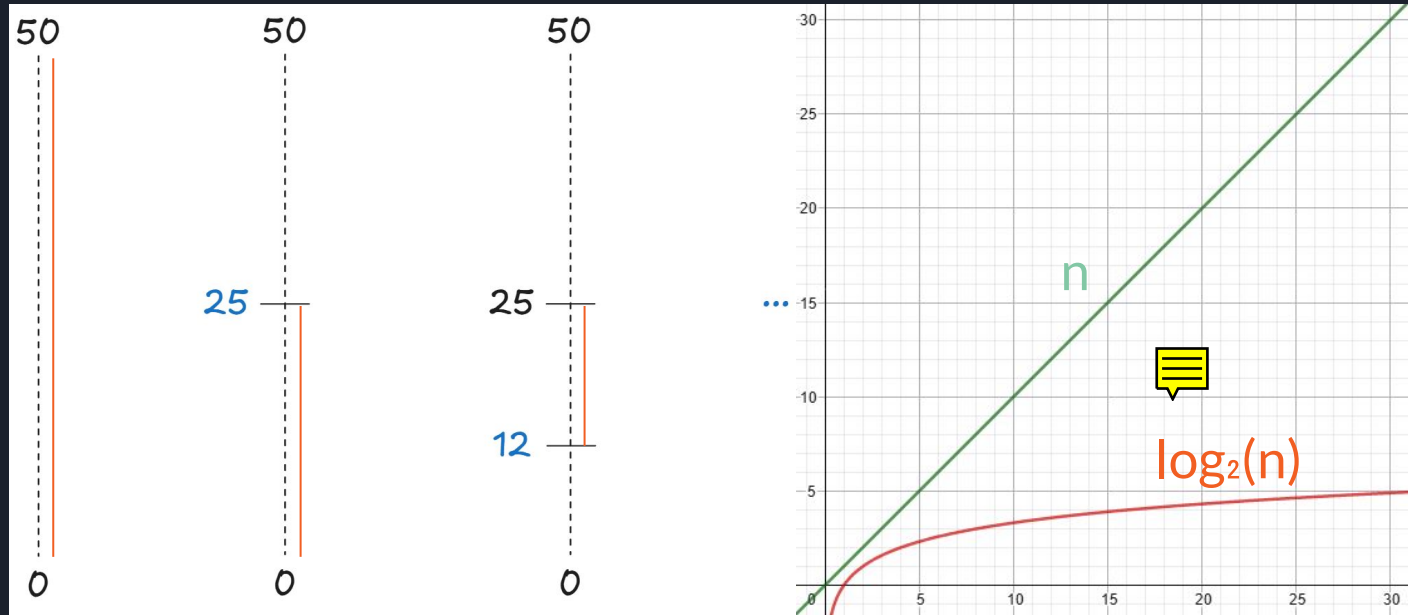
La recherche dichotomique est un `algorithme de recherche` pour trouver la `position` d'un `élément` dans un tableau `trié`.

Principe

Élément recherché : 14



Complexité de la recherche dichotomique





Exemple

-12	-1	0	2	3	5	10	12	15	16	17	25	31	36	42
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Exemple

Nombre cherché : 8 

8 															
-12	-1	0	2	3	5	10	12	15	16	17	25	31	36	42	
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	



Exemple

Nombre cherché : 8

start = 0

end = 14

-12	-1	0	2	3	5	10	12	15	16	17	25	31	36	42
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
start														end



Exemple

Nombre cherché : 8

start = 0

start <= end ? Oui

end = 14

-12	-1	0	2	3	5	10	12	15	16	17	25	31	36	42
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14



Exemple

Nombre cherché : 8

start = 0

start <= end ? Oui

half = $\lfloor (\text{start} + \text{end}) / 2 \rfloor = 7$

end = 14

Nombre trouvé ? Non

-12	-1	0	2	3	5	10	12	15	16	17	25	31	36	42
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14



Exemple

Nombre cherché : 8 (avec $8 < 12$)

start = 0

end = 6

-12	-1	0	2	3	5	10	12	15	16	17	25	31	36	42
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14



Exemple

Nombre cherché : 8

start = 0

start <= end ? Oui

end = 6

-12	-1	0	2	3	5	10	12	15	16	17	25	31	36	42
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Exemple

Nombre cherché : 8

start = 0

start <= end ? Oui

half = $\lfloor (\text{start} + \text{end}) / 2 \rfloor = 3$

end = 6

Nombre trouvé ? Non

-12	-1	0	2	3	5	10	12	15	16	17	25	31	36	42
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14



Exemple

Nombre cherché : 8 (avec $8 > 2$)

start = 4

end = 6

-12	-1	0	2	3	5	10	12	15	16	17	25	31	36	42
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Exemple

Nombre cherché : 8

start = 4

start <= end ? Oui

end = 6

-12	-1	0	2	3	5	10	12	15	16	17	25	31	36	42
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Exemple

Nombre cherché : 8

start = 4

start <= end ? Oui

half = $\lfloor (\text{start} + \text{end}) / 2 \rfloor = 5$

end = 6

Nombre trouvé ? Non

-12	-1	0	2	3	5	10	12	15	16	17	25	31	36	42
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Exemple

Nombre cherché : 8 (avec $8 > 5$)

start = 6

end = 6

-12	-1	0	2	3	5	10	12	15	16	17	25	31	36	42
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14



Exemple

Nombre cherché : 8

start = 6

start <= end ? Oui

end = 6

-12	-1	0	2	3	5	10	12	15	16	17	25	31	36	42
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Exemple

Nombre cherché : 8

start = 6

start <= end ? Oui

half = $\lfloor (\text{start} + \text{end}) / 2 \rfloor = 6$

end = 6

Nombre trouvé ? Non

-12	-1	0	2	3	5	10	12	15	16	17	25	31	36	42
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Exemple

Nombre cherché : 8 (avec $8 < 10$)

start = 6

end = 5

-12	-1	0	2	3	5	10	12	15	16	17	25	31	36	42
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14



Exemple

Nombre cherché : 8

start = 6

start <= end ? Non

end = 5

→ 8 n'est pas dans le tableau

-12	-1	0	2	3	5	10	12	15	16	17	25	31	36	42
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

De recherche à insertion

Nombre à insérer : 8 

start = 6

start <= end ? **Non**

end = 5

-12	-1	0	2	3	5	10	12	15	16	17	25	31	36	42
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

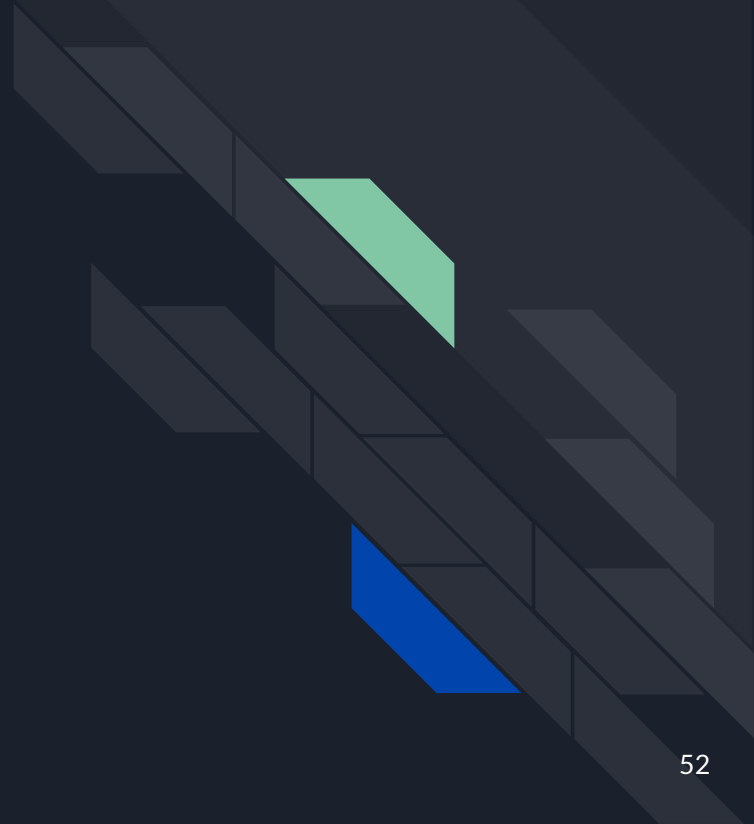
Nombre de comparaisons pour insérer

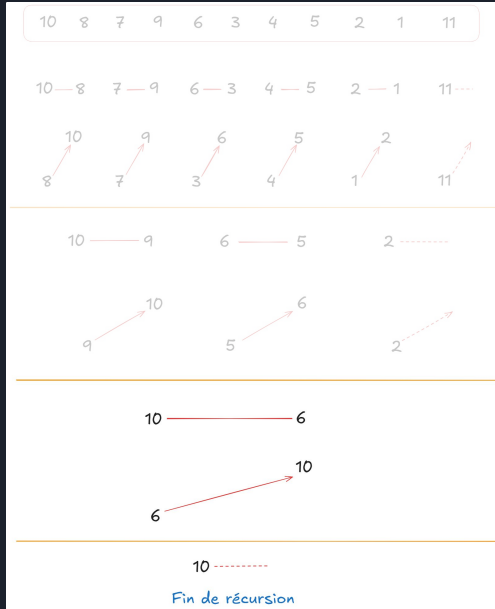
n	1	2	3	4	5	6	7
comparaisons $\lfloor \log_2(n) + 1 \rfloor$	1	2	2	3	3	3	3



n	8	9	...	15	16	...	31	32	...	63	64	...
comparaisons $\lfloor \log_2(n) + 1 \rfloor$	4	4	4	4	5	5	5	6	6	6	7	7

Insertion



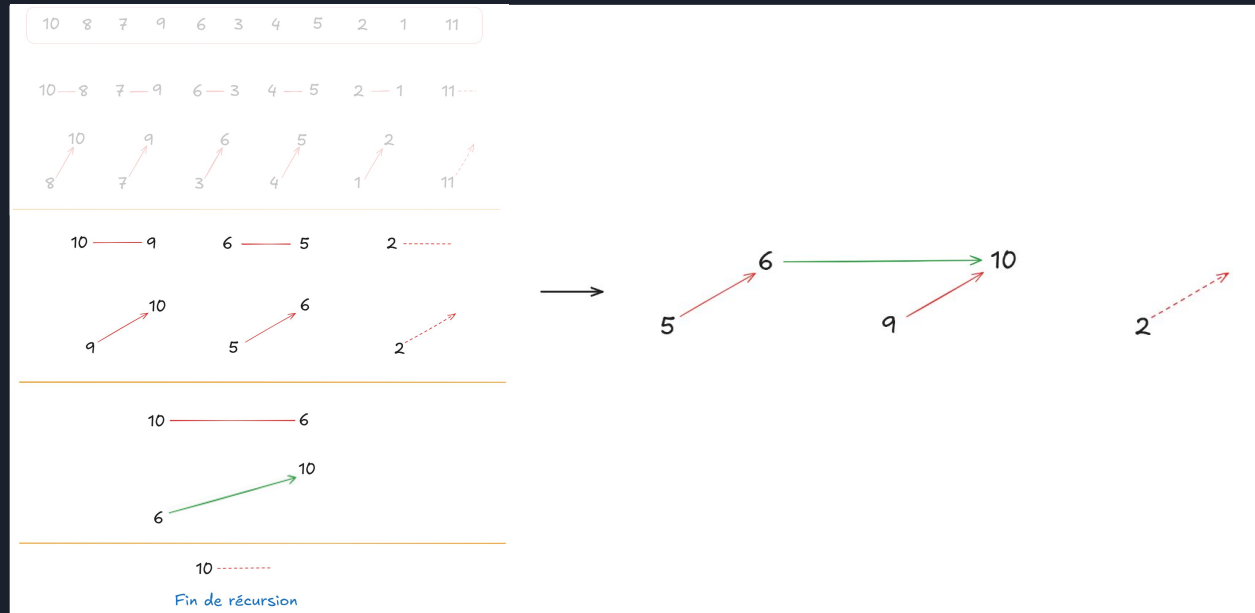
étape de

$n < 3$?

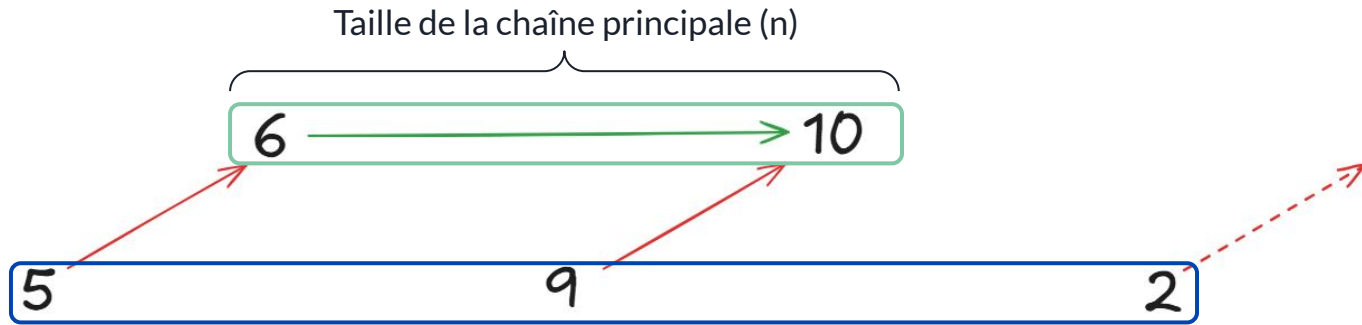
Éléments déjà triés

Organisation actuelle des éléments

étape de
réursion: 1



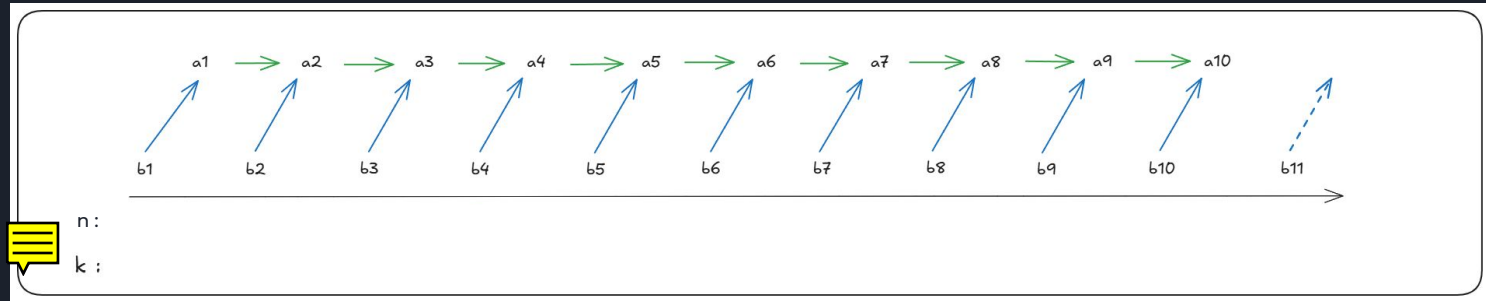
Notation et vocabulaire



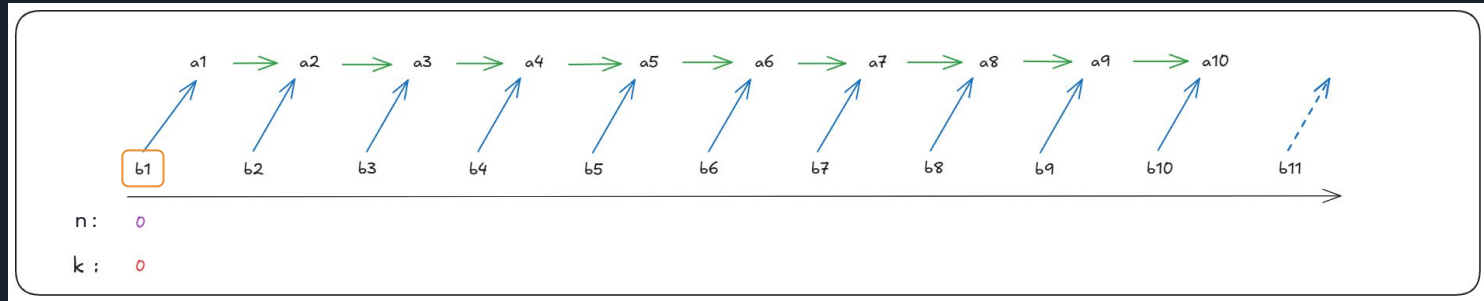
Chaîne principale (main chain) → chaîne triée

Éléments à insérer → perdants

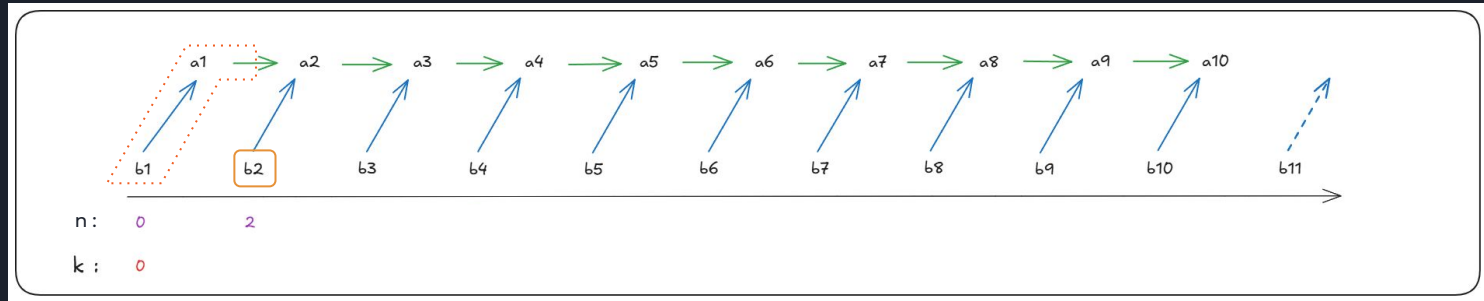
Différents ordres d'insertion



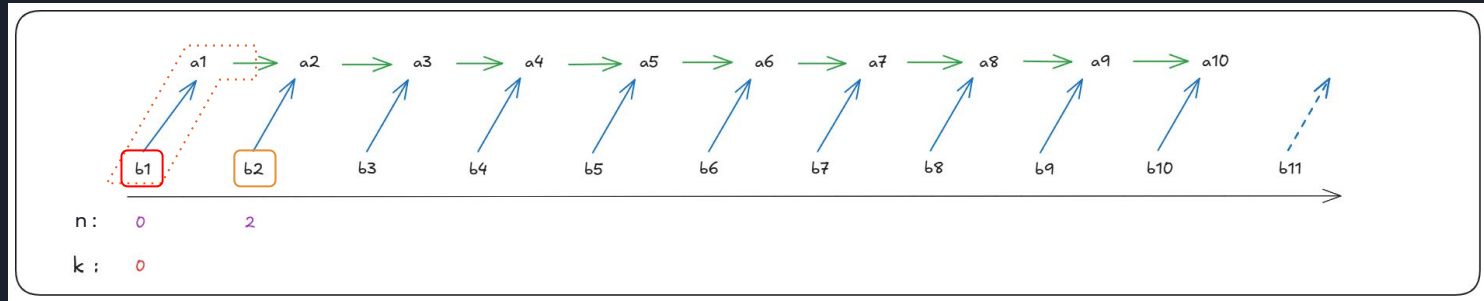
Différents ordres d'insertion



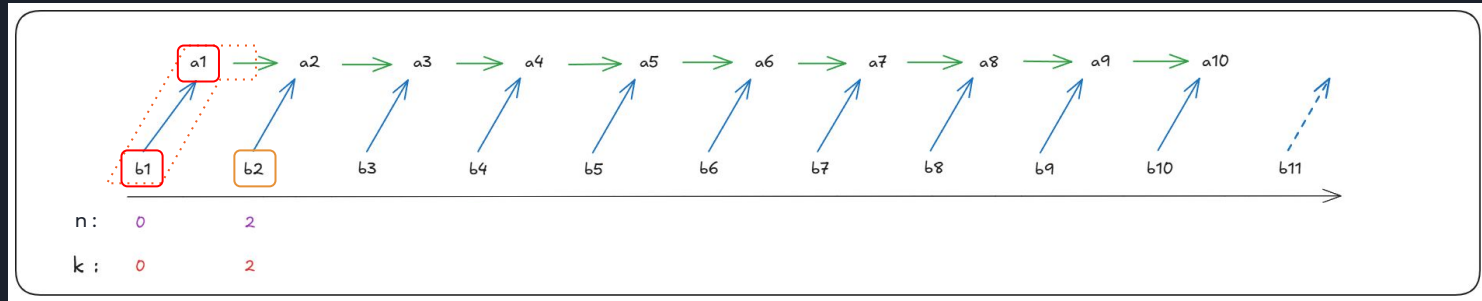
Différents ordres d'insertion



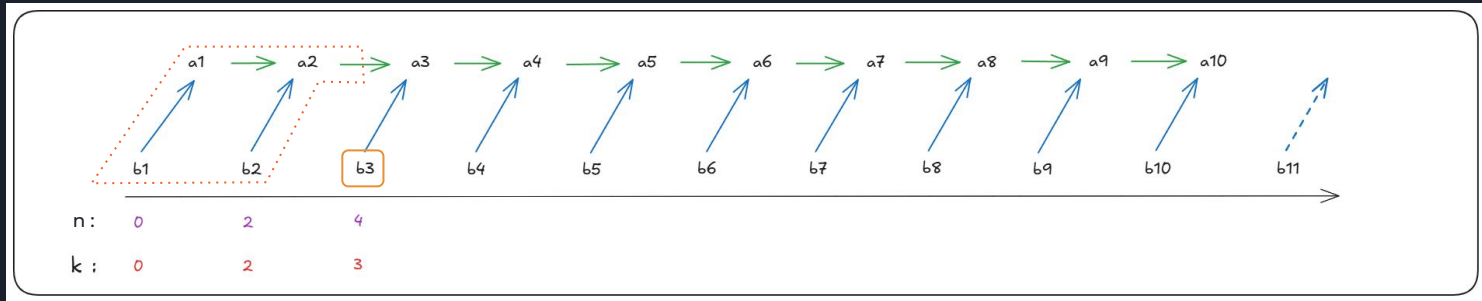
Différents ordres d'insertion



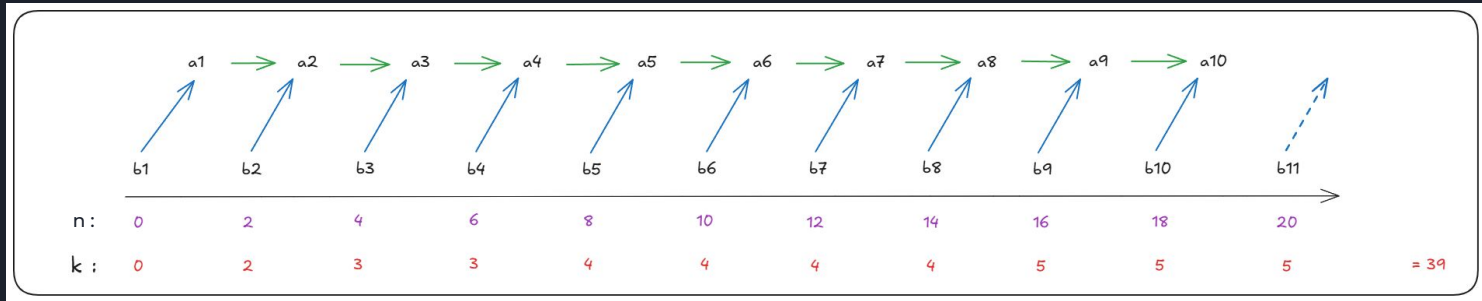
Différents ordres d'insertion



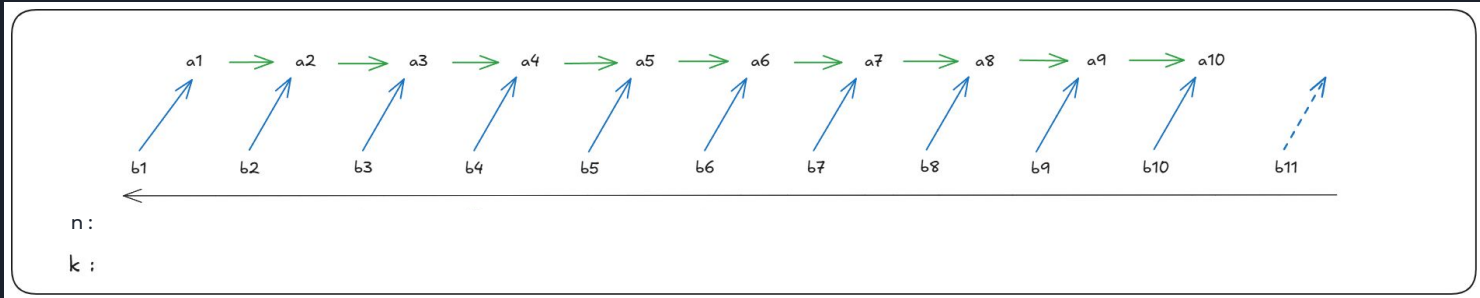
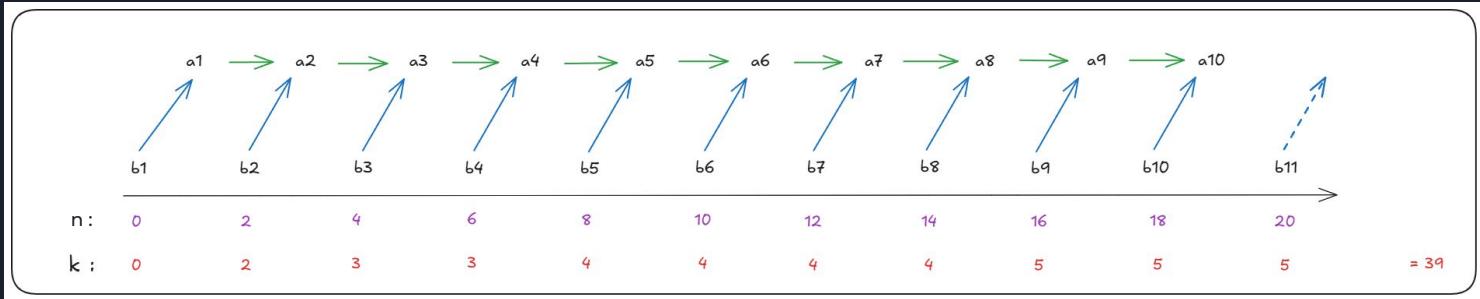
Différents ordres d'insertion



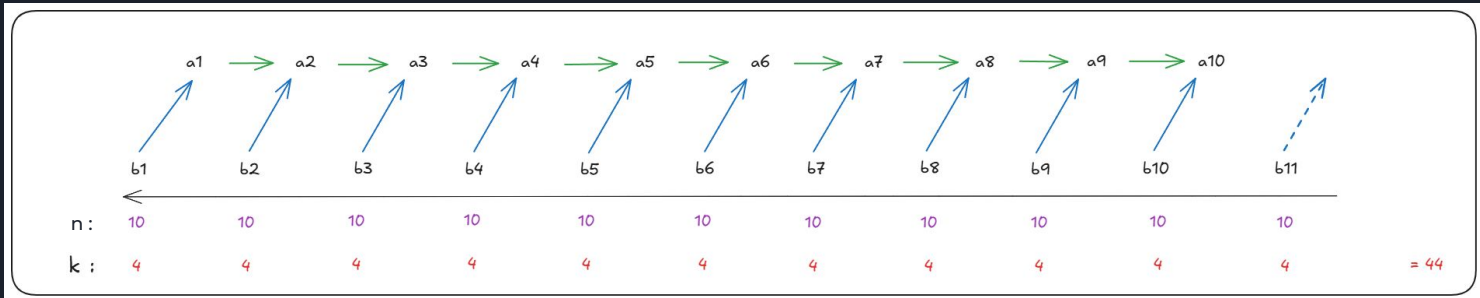
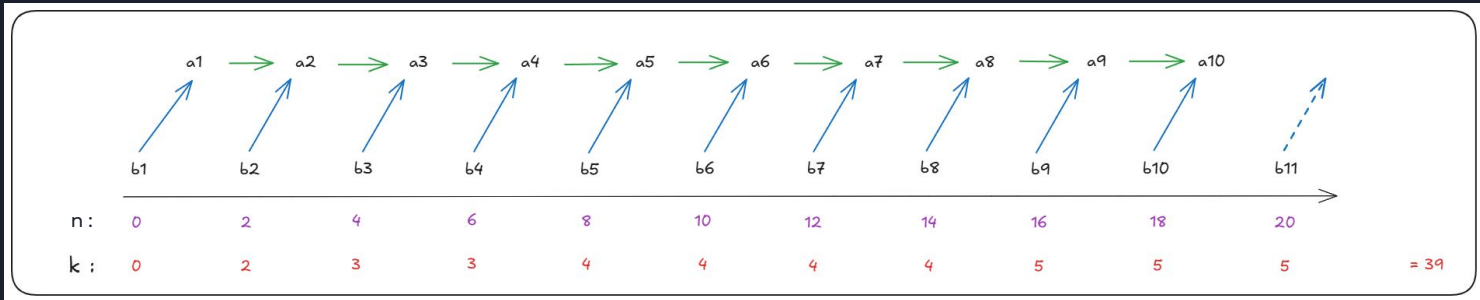
Différents ordres d'insertion



Différents ordres d'insertion



Différents ordres d'insertion





Nombre de comparaisons pour insérer

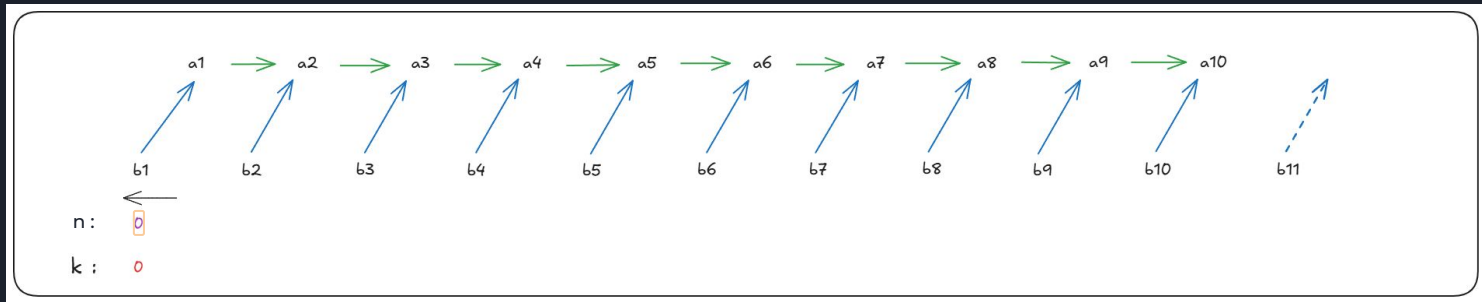
n	1	2	3	4	5	6	7
comparaisons $\lfloor \log_2(n) + 1 \rfloor$	1	2	2	3	3	3	3

n	8	9	...	15	16	...	31	32	...	63	64	...
comparaisons $\lfloor \log_2(n) + 1 \rfloor$	4	4	4	4	5	5	5	6	6	6	7	7

$$N = \{1, 3, 7, 15, \dots\}$$

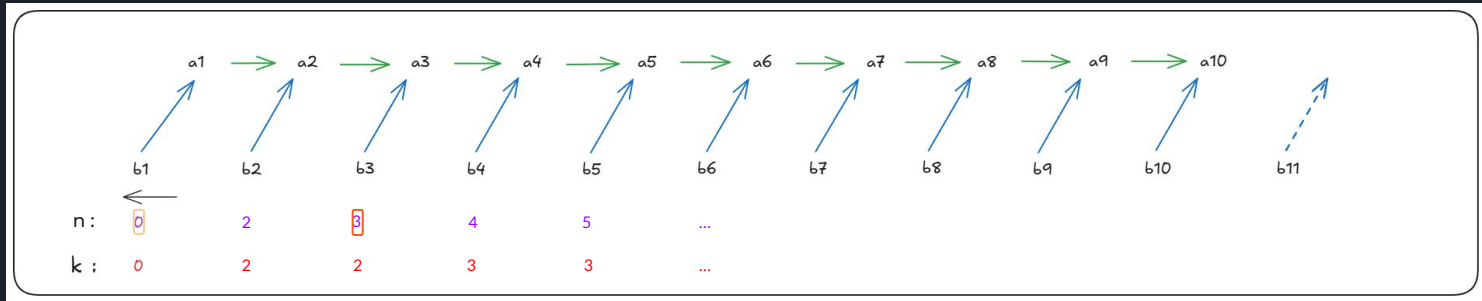
Optimiser l'insertion

$N = \{1, 3, 7, 15, \dots\}$



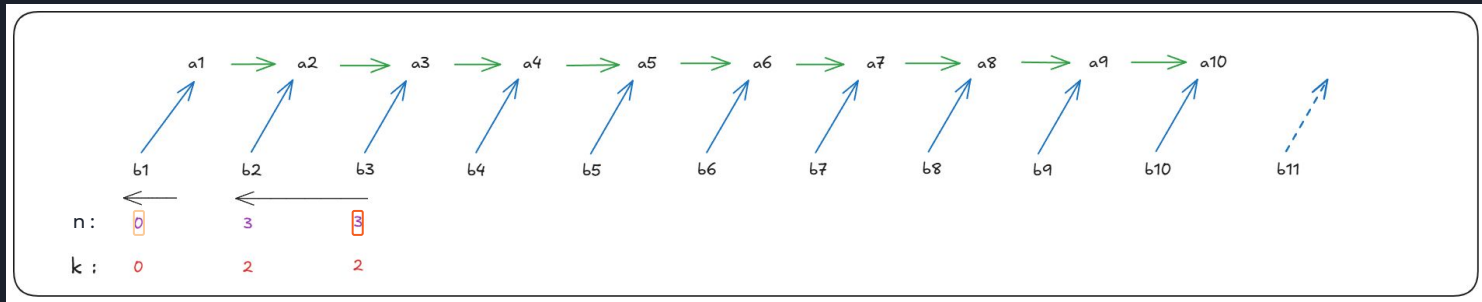
Optimiser l'insertion

$N = \{1, 3, 7, 15, \dots\}$



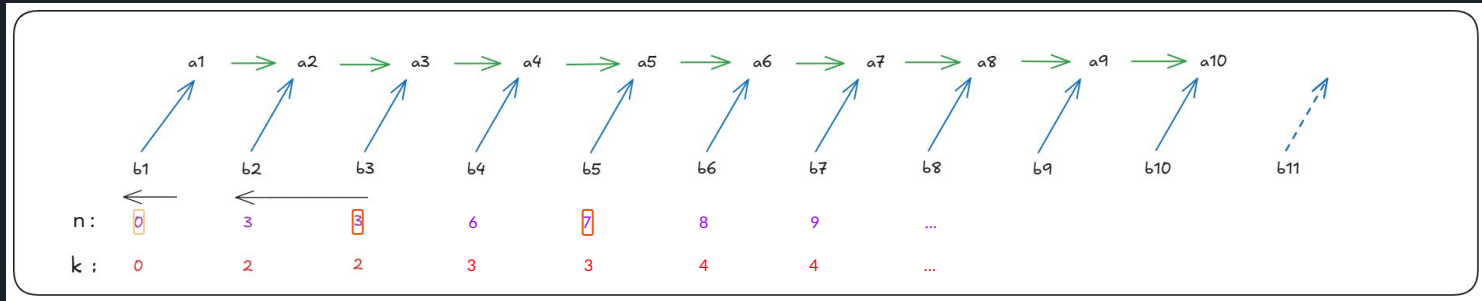
Optimiser l'insertion

$N = \{1, 3, 7, 15, \dots\}$



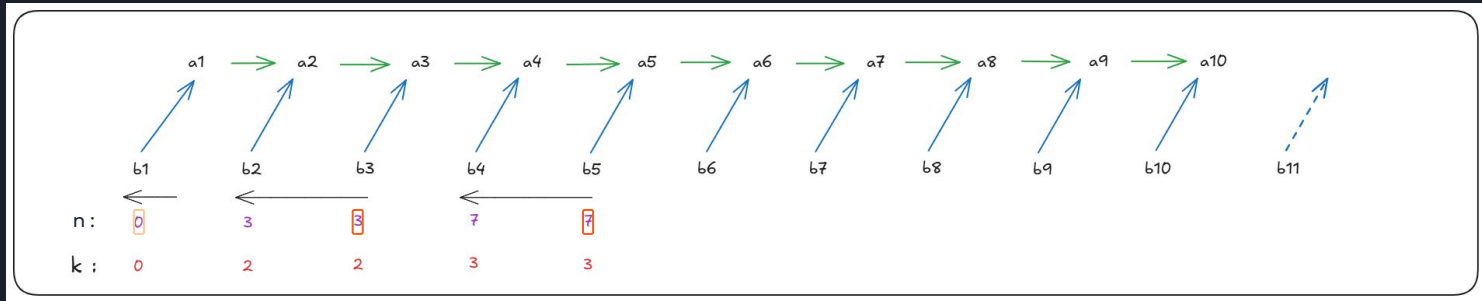
Optimiser l'insertion

$N = \{1, 3, 7, 15, \dots\}$



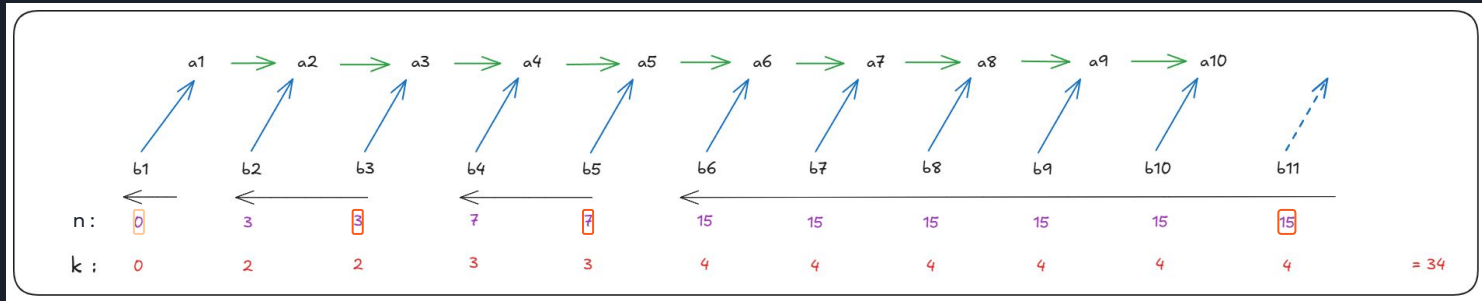
Optimiser l'insertion

$N = \{1, 3, 7, 15, \dots\}$



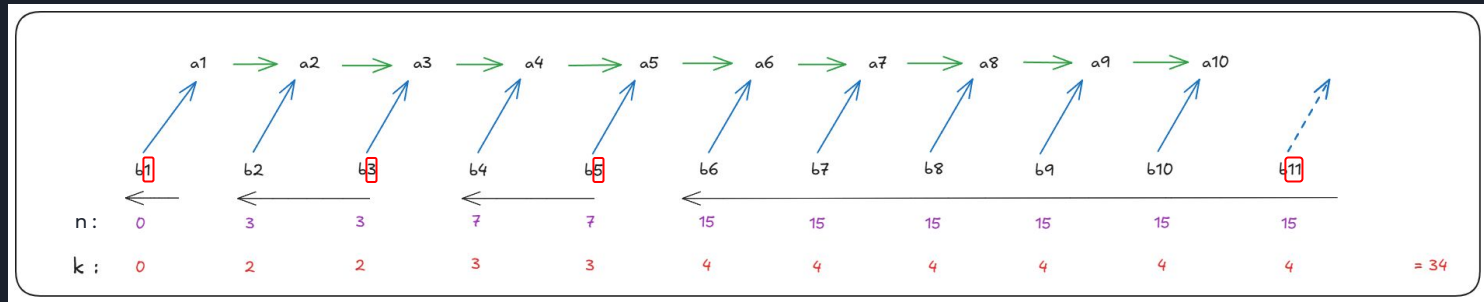
Optimiser l'insertion

$N = \{1, 3, 7, 15, \dots\}$



Optimiser l'insertion

$N = \{1, 3, 7, 15, \dots\}$



$$t_1 = 1$$

$$t_2 = 3$$

$$t_3 = 5$$

$$t_4 = 11$$



Suite de Jacobsthal

$\{0, 1, 1, 3, 5, 11, 21, 43, 85, \dots\}$

$$\begin{aligned} J_0 &= 0 \\ J_1 &= 1 \\ J_n &= J_{n-1} + 2 J_{n-2} \end{aligned}$$

$$J_{n+1} = 2^n - J_n$$

$$J_n = \frac{2^n - (-1)^n}{3}$$

Suite de Jacobsthal

{0, 1, 1, 3, 5, 11, 21, 43, 85, ...}

$$\begin{aligned} J_0 &= 0 \\ J_1 &= 1 \\ J_n &= J_{n-1} + 2J_{n-2} \end{aligned}$$

$$J_{n+1} = 2^n - J_n$$

$$J_n = \frac{2^n - (-1)^n}{3}$$



{1, 1, 3, 5, 11, 21, 43, 85, ...}

$$\begin{aligned} t_0 &= 1 \\ t_1 &= 1 \\ t_k &= t_{k-1} + 2t_{k-2} \end{aligned}$$

$$t_k = 2^{k-1} - t_{k-1}$$

$$t_k = \frac{2^{k+1} + (-1)^k}{3}$$



Taille de la chaîne principale

Soit t_k le nombre de Jacobsthal actuel et t_{k-1} le nombre précédent :

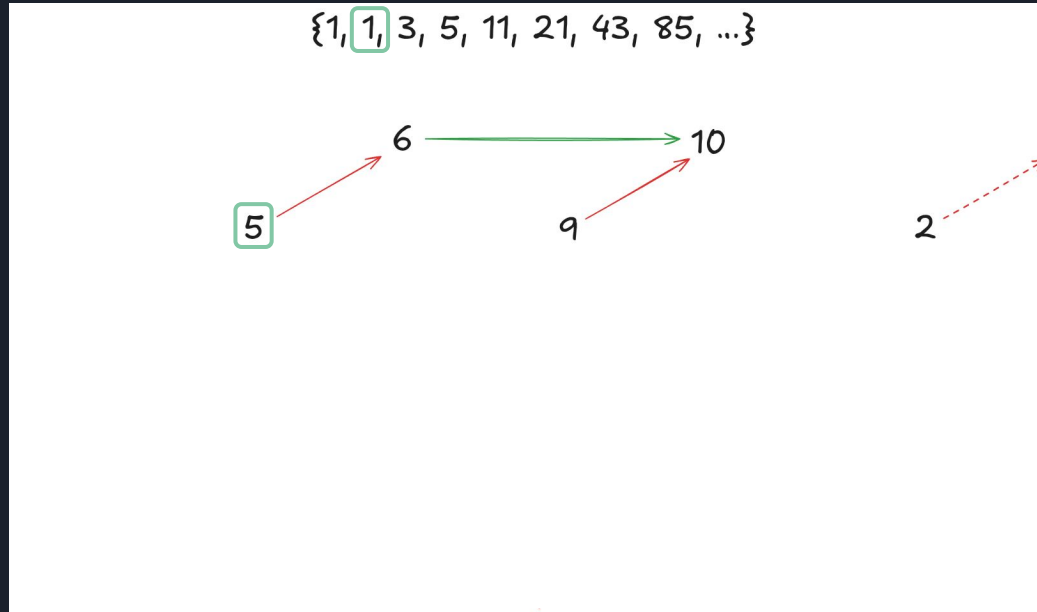
$$n = 2 * t_{k-1} + (t_k - t_{k-1} - 1)$$

Donc $n = t_{k-1} + t_k - 1$



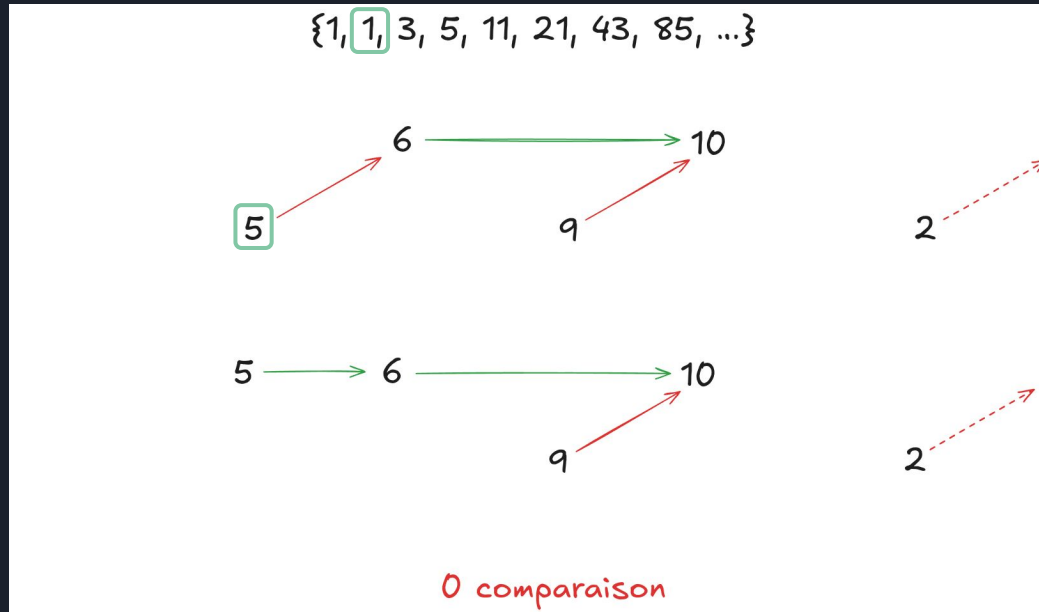
Insérer les éléments

étape de
réursion : 1



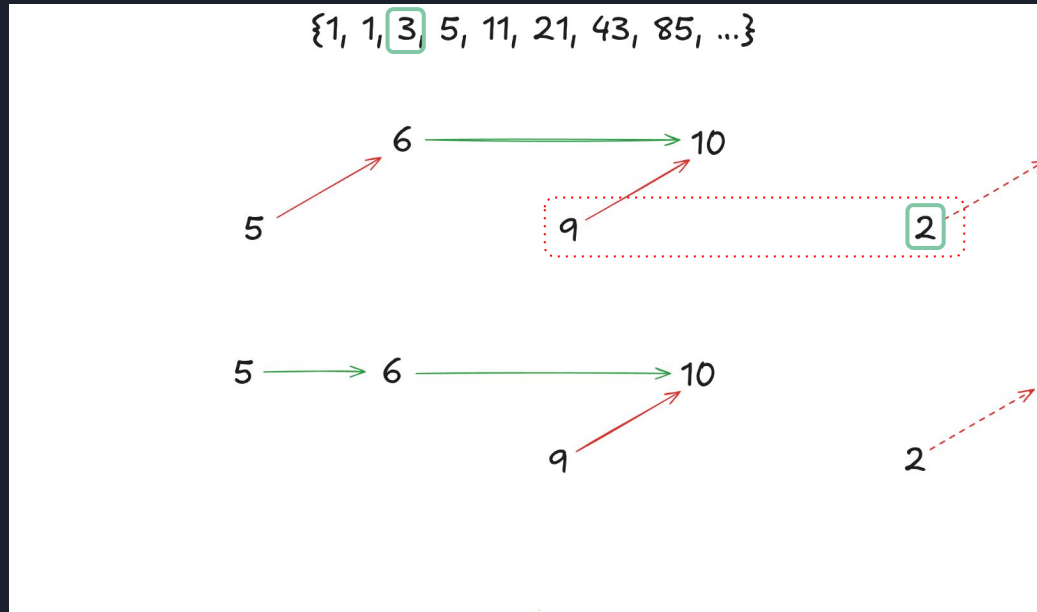
Insérer les éléments

étape de
réursion : 1



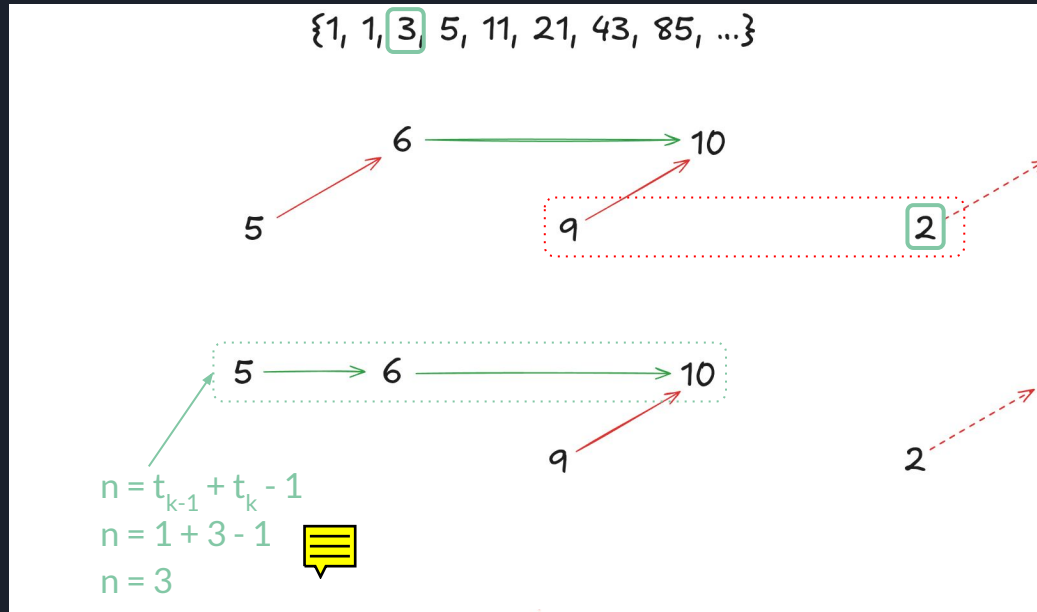
Insérer les éléments

étape de
réursion : 1



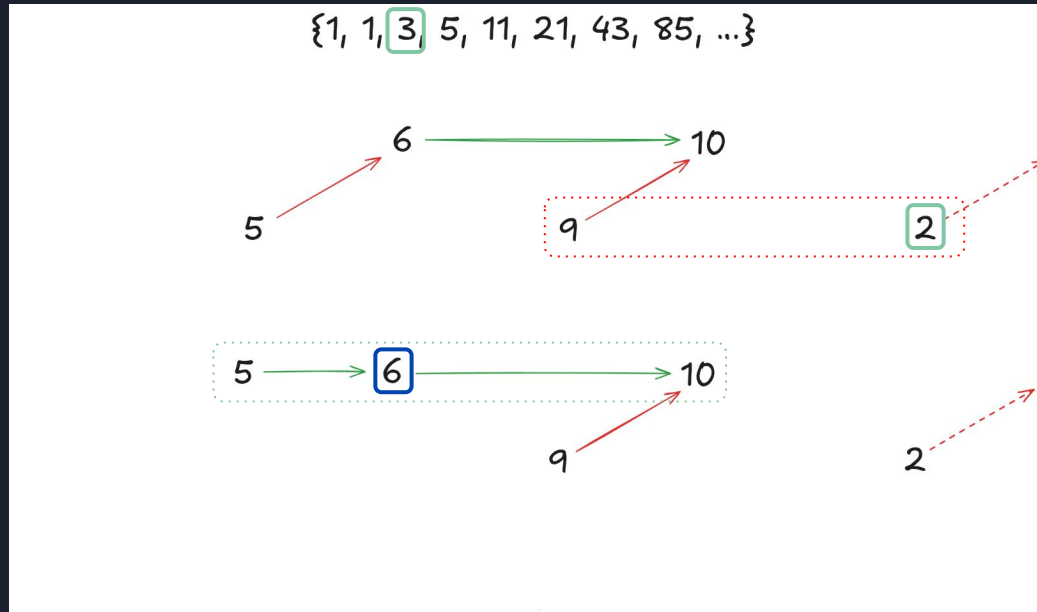
Insérer les éléments

étape de
réursion : 1



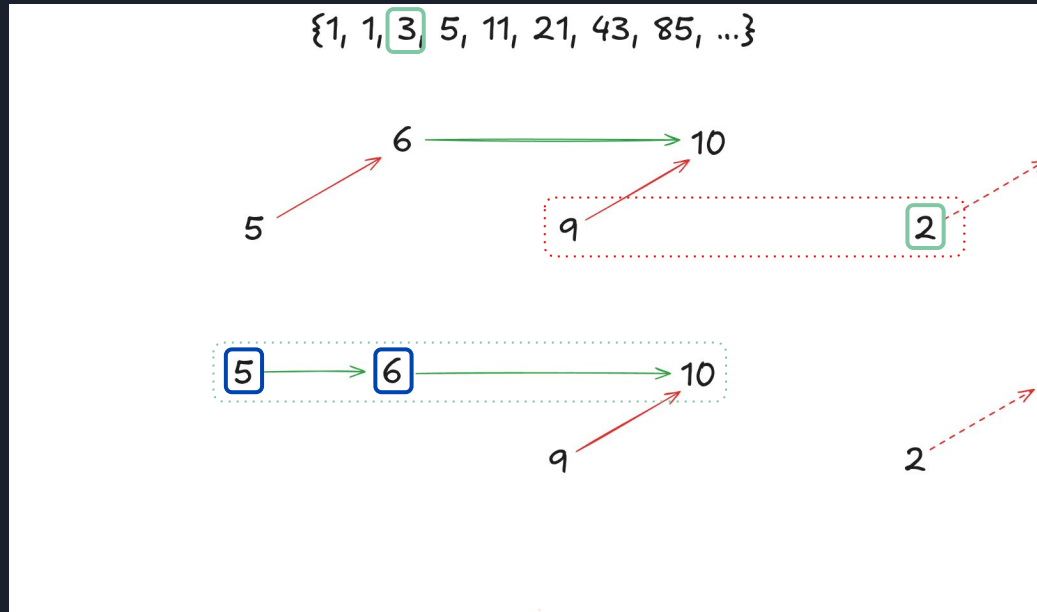
Insérer les éléments

étape de
réursion : 1



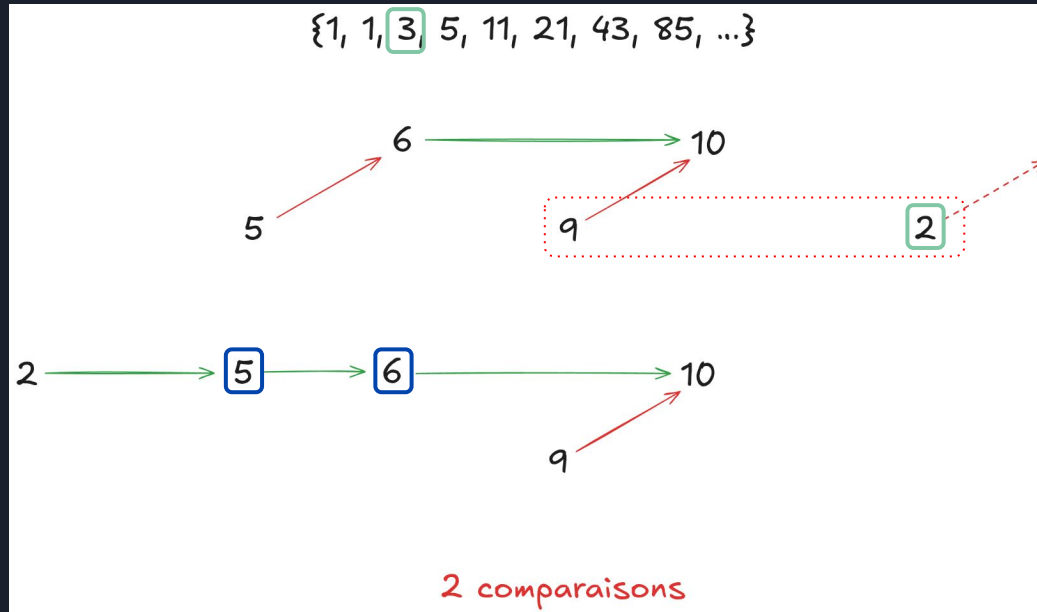
Insérer les éléments

étape de
réursion : 1



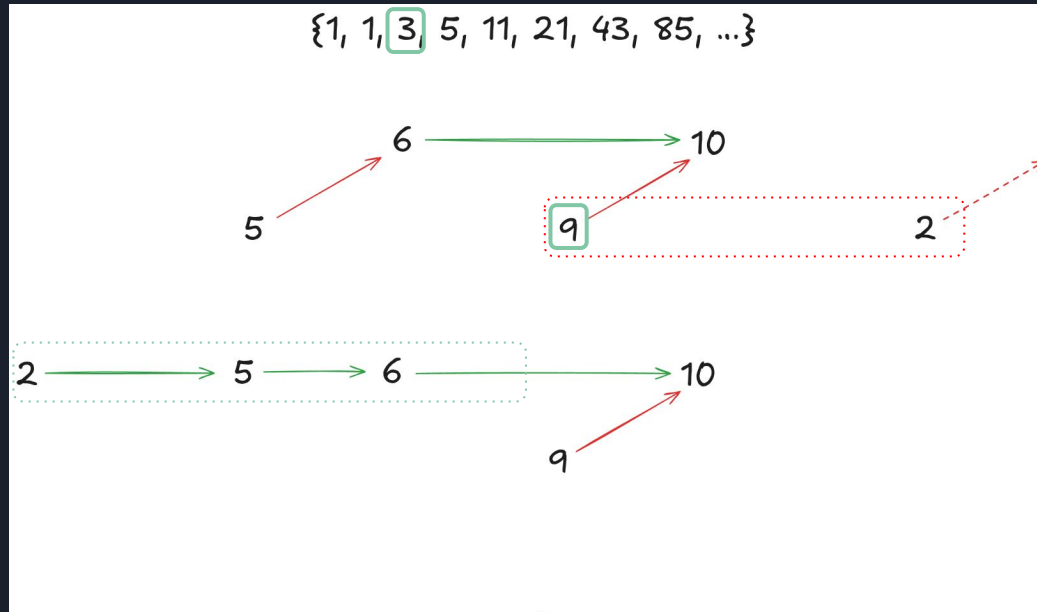
Insérer les éléments

étape de
réursion : 1



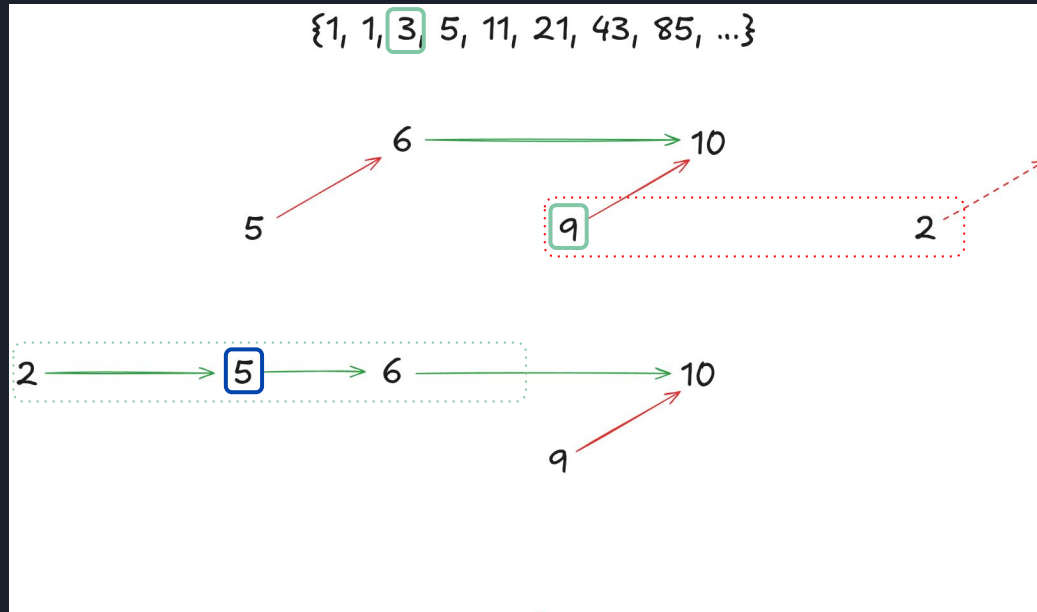
Insérer les éléments

étape de
réursion : 1



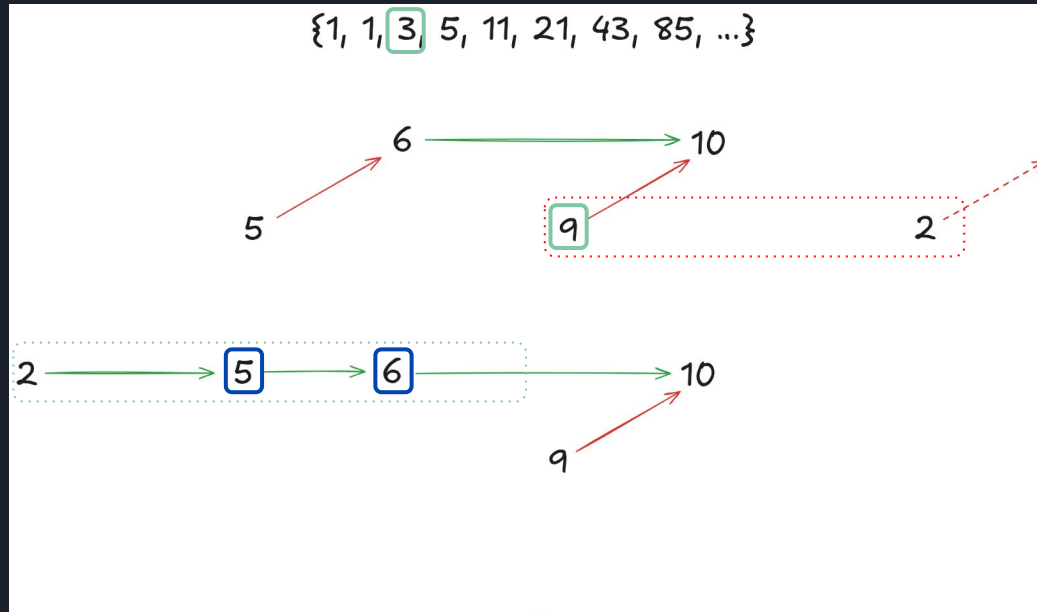
Insérer les éléments

étape de
réursion : 1



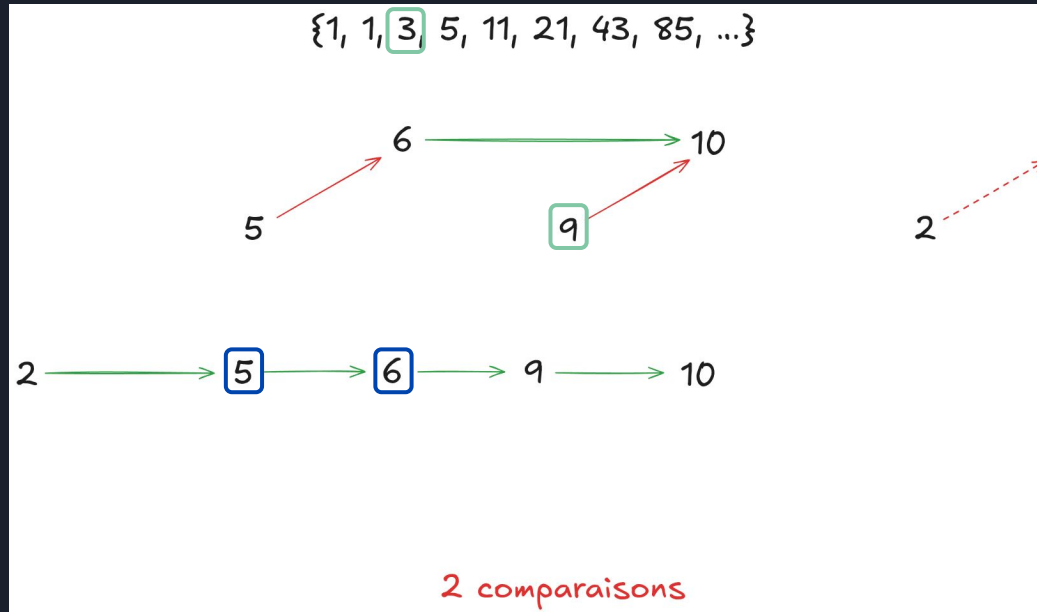
Insérer les éléments

étape de
réursion : 1



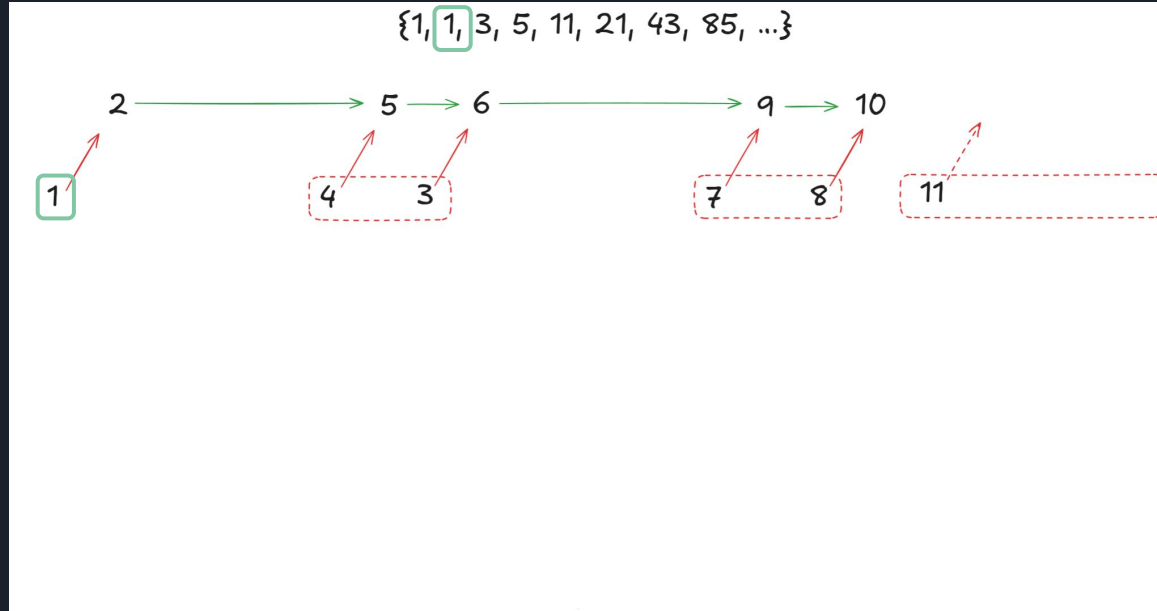
Insérer les éléments

étape de
réursion : 1



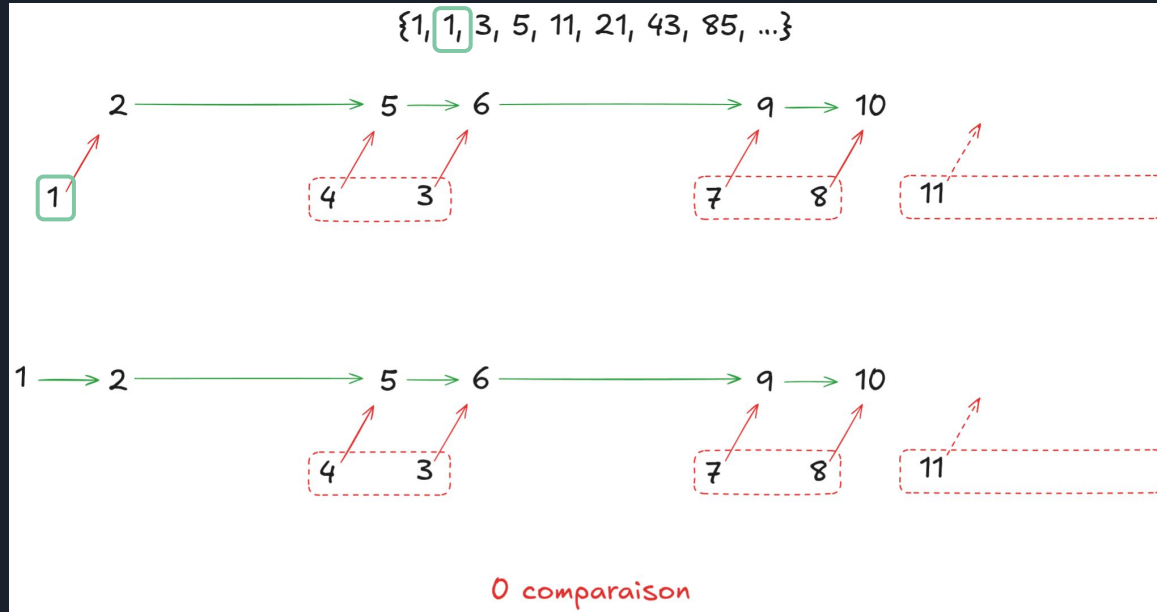
Insérer les éléments

étape de
réursion : 0



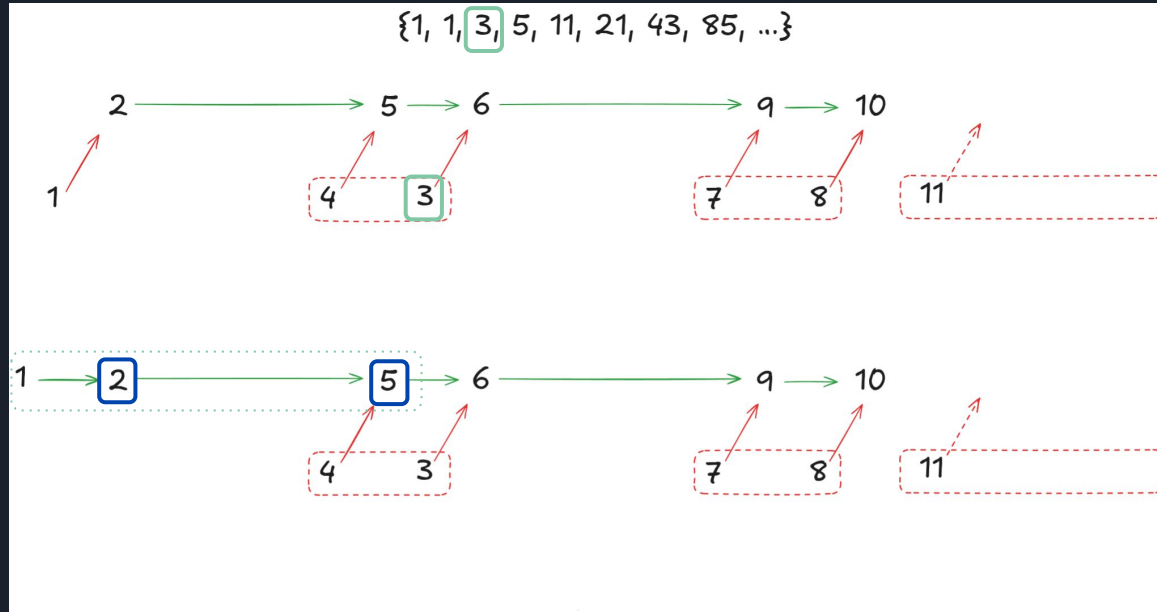
Insérer les éléments

étape de
réursion: 0



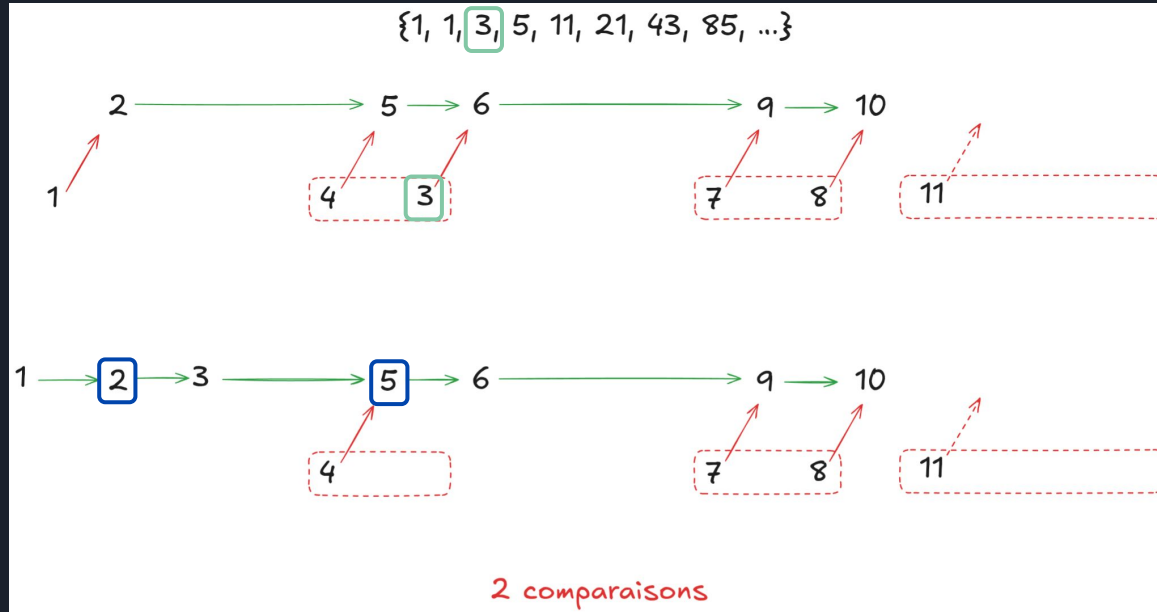
Insérer les éléments

étape de
réursion: 0



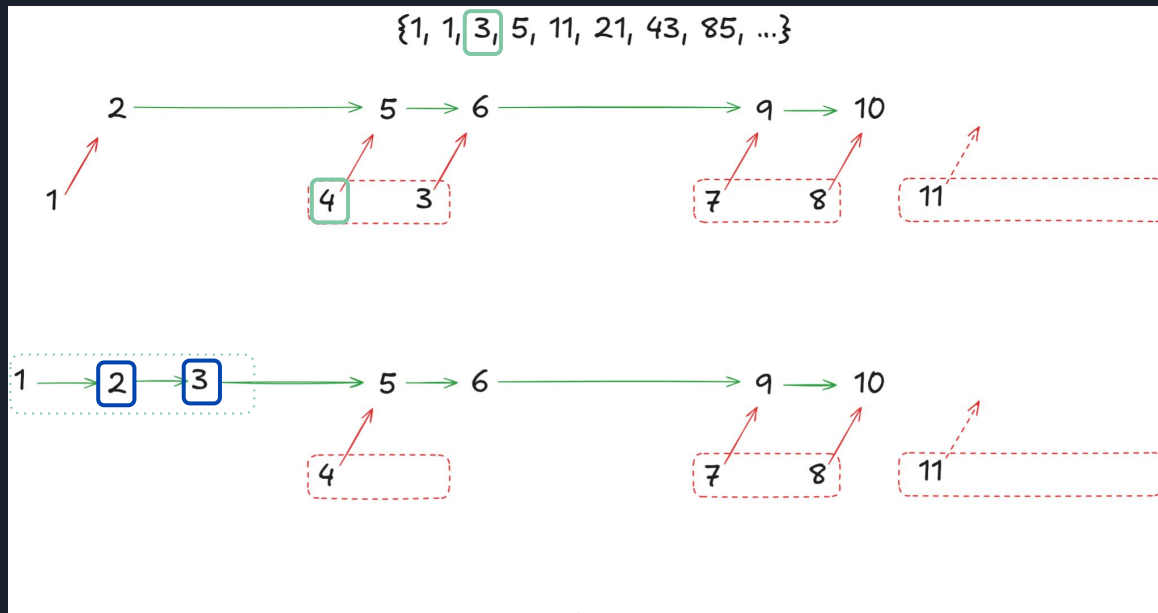
Insérer les éléments

étape de
réursion: 0



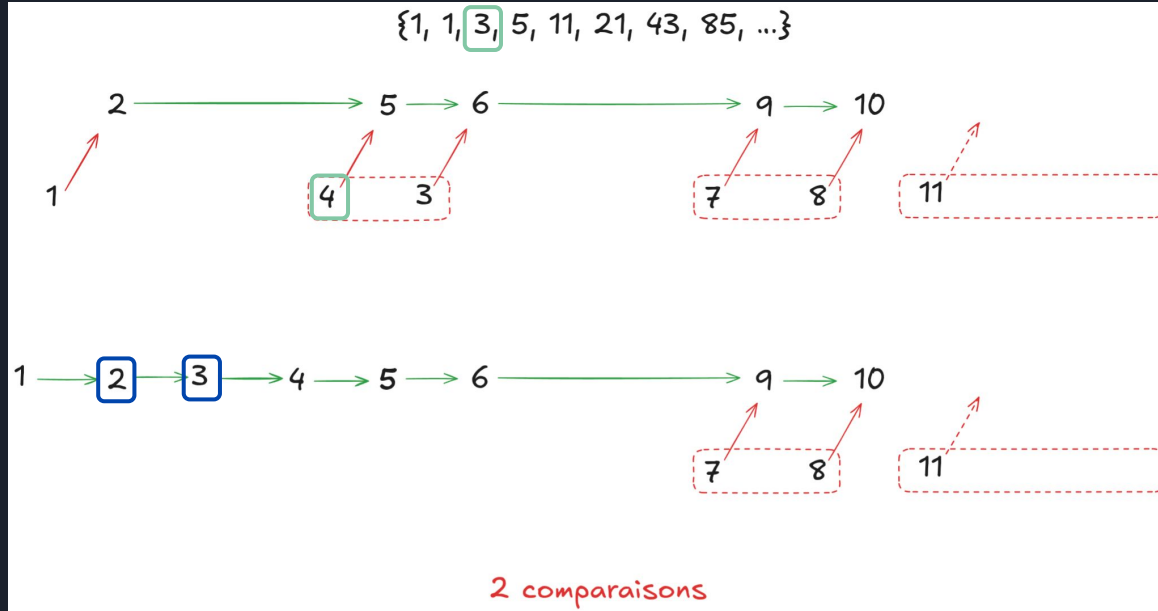
Insérer les éléments

étape de
réursion: 0



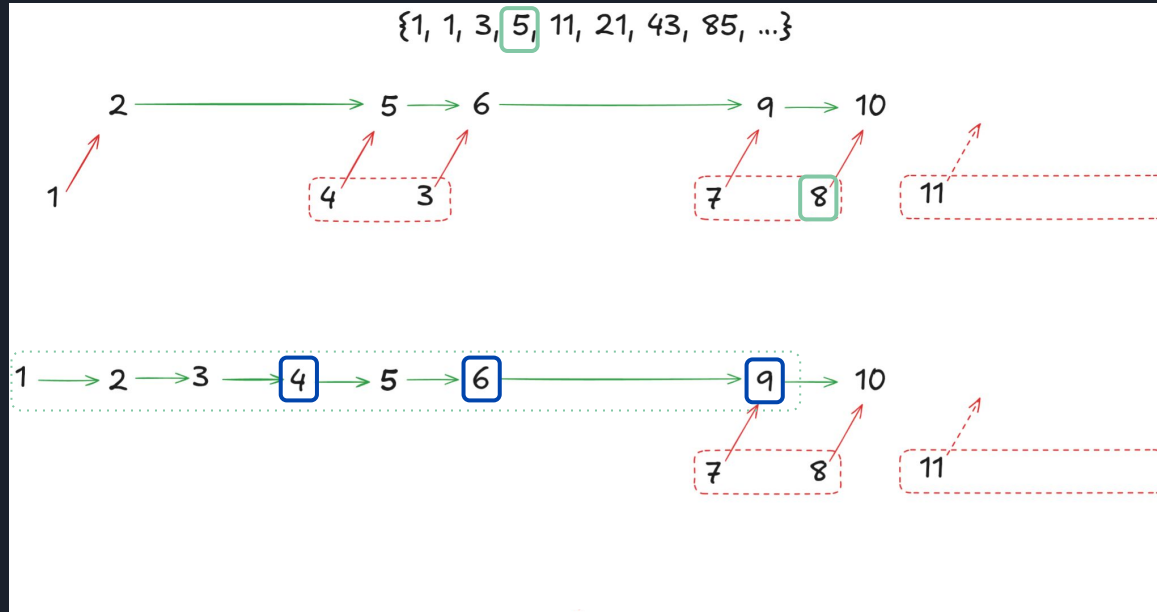
Insérer les éléments

étape de
réursion: 0



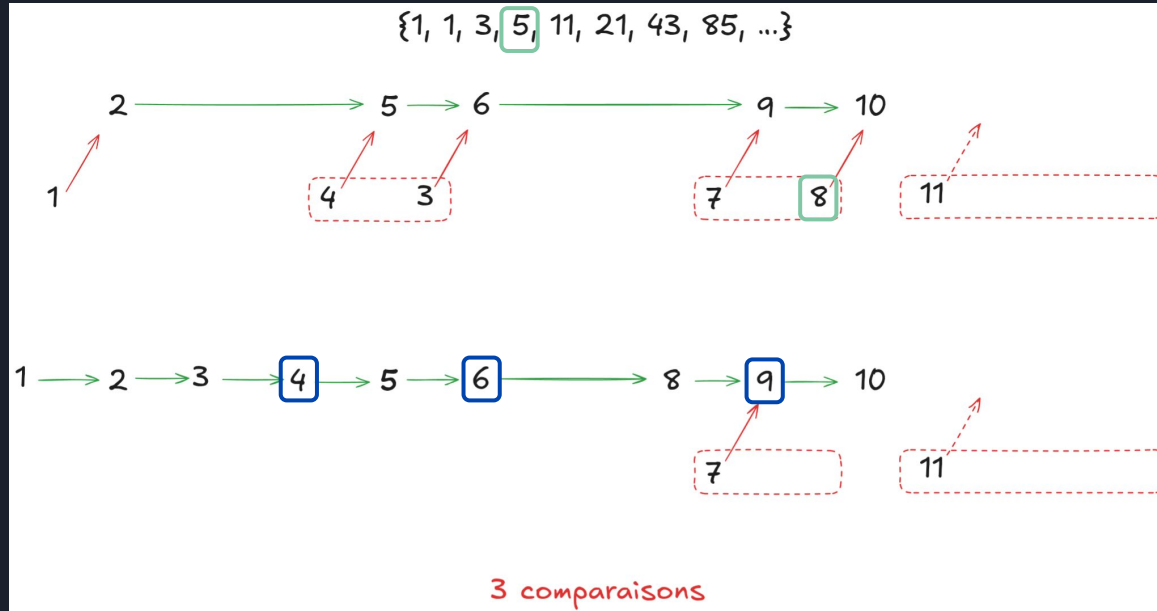
Insérer les éléments

étape de
réursion: 0



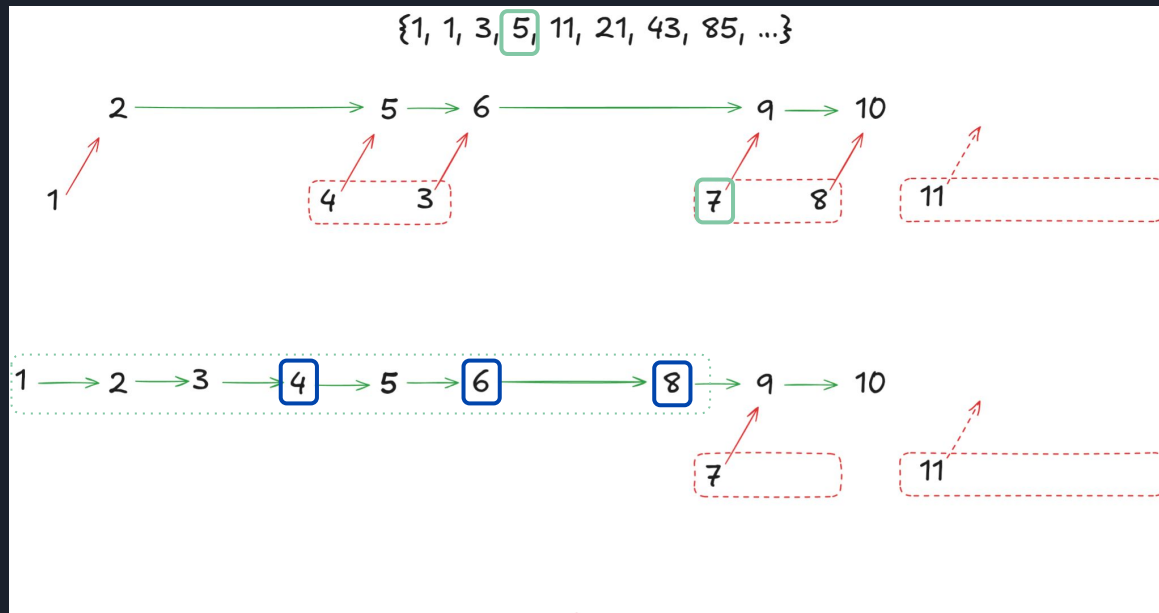
Insérer les éléments

étape de
réursion: 0



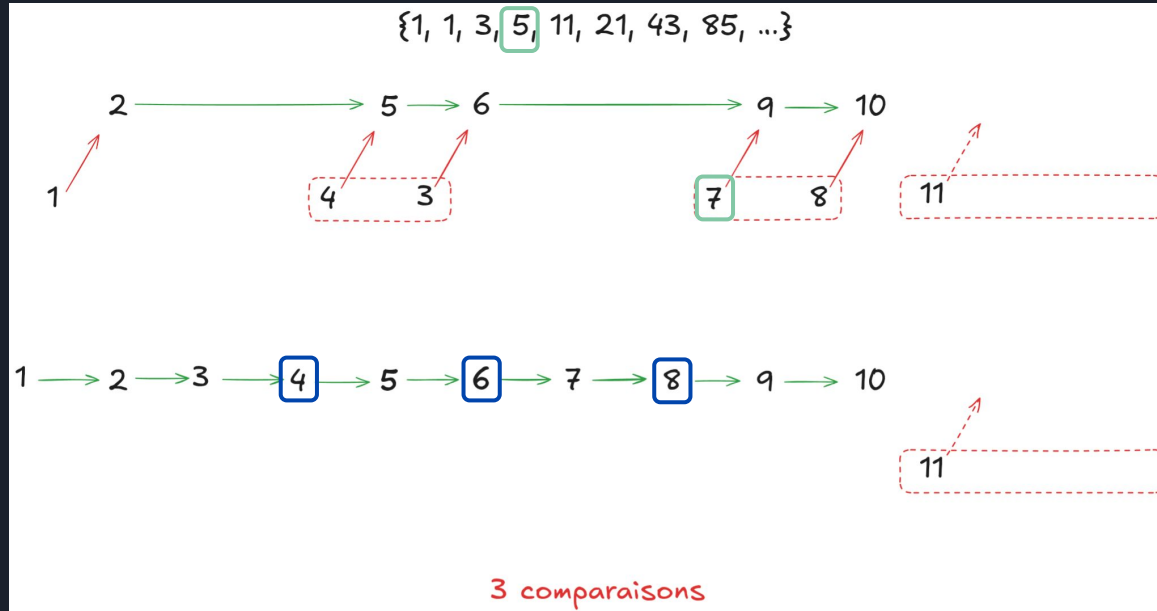
Insérer les éléments

étape de
réursion: 0



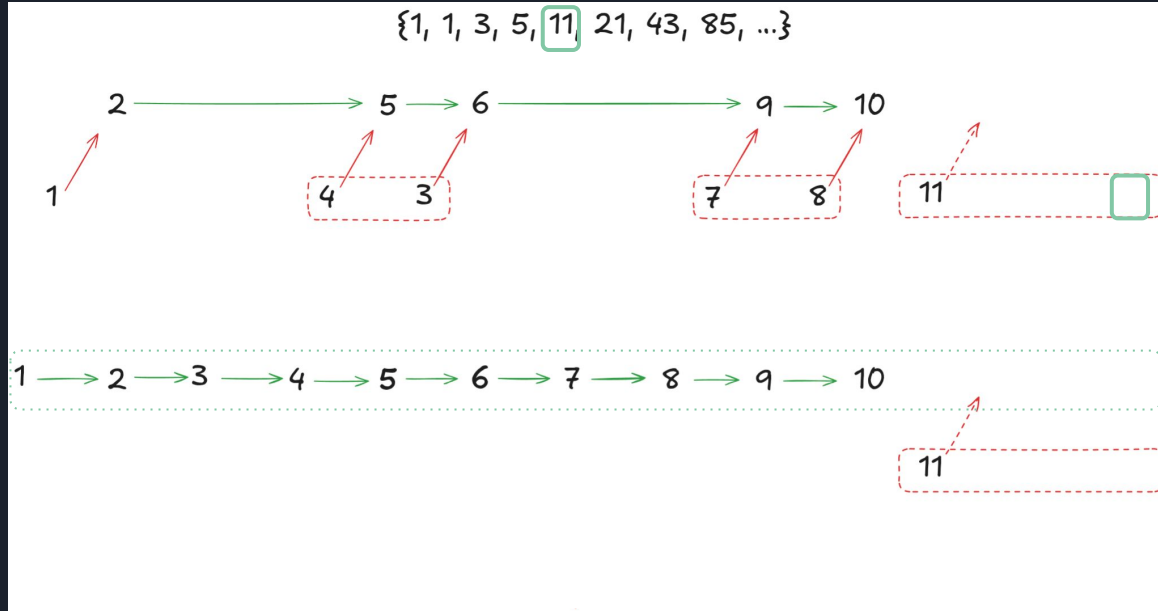
Insérer les éléments

étape de
réursion : 0



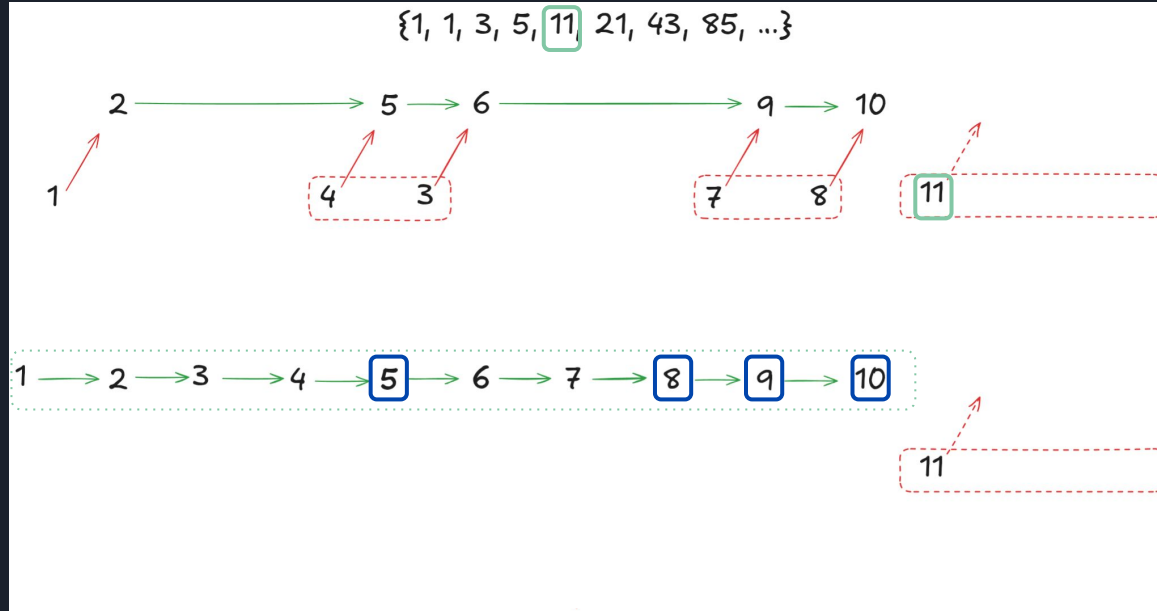
Insérer les éléments

étape de
réursion: 0



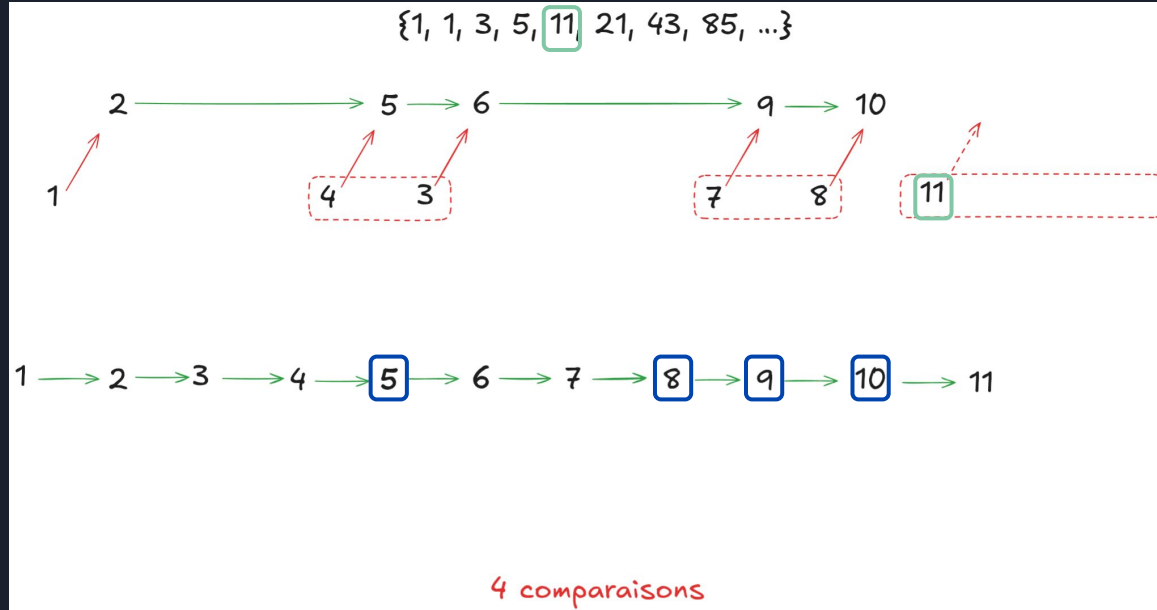
Insérer les éléments

étape de
réursion: 0



Insérer les éléments

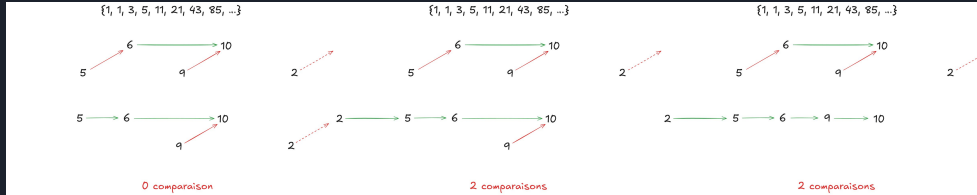
étape de
réursion: 0



Nombre de comparaisons pour l'insertion

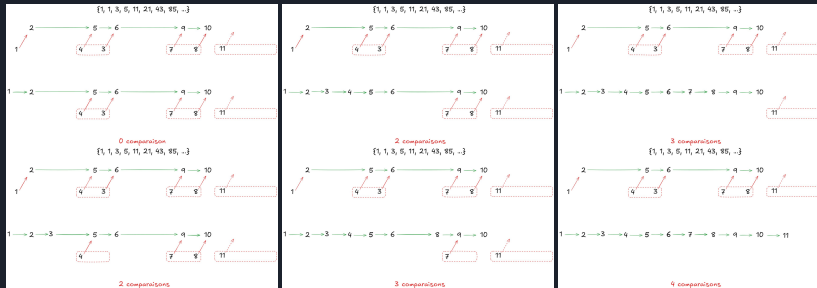
étape de récursion

1



$0 + 2 + 2 =$
4 comparaisons

0



$0 + 2 + 2 + 3 + 3 + 4 =$
14 comparaisons

18 comparaisons au total



Formule du nombre de comparaisons de l'insertion

Soit m le nombre de perdants.

Soit k l'indice du nombre de Jacobsthal tel que t_k soit le plus petit nombre possible de la suite respectant $t_k \geq m$.

$$G(m) = k * m - \sum_{j=0}^{k-1} t_j \text{ comparaisons par étape d'insertion}$$

$$G(m) = k * m - \lfloor 2^{k+1}/3 \rfloor$$

Exemple: 

$$G(3) = 3 * 3 - \lfloor 2^{3+1}/3 \rfloor = 9 - \lfloor 16/3 \rfloor = 9 - 5 = 4$$

$$G(6) = 4 * 6 - \lfloor 2^{4+1}/3 \rfloor = 24 - \lfloor 32/3 \rfloor = 24 - 10 = 14$$

Formule du nombre de comparaisons

$$F(n) = \lfloor n/2 \rfloor + F(\lfloor n/2 \rfloor) + G(\lceil n/2 \rceil)$$

Merge Récurrence Insertion

$$F(n) = \sum_{k=1}^n \lceil \log_2(3/4 * k) \rceil = \sum_{k=1}^n \lceil \log_2(0.75 * k) \rceil$$

Exemple : 

$$F(11) = \sum_{k=1}^{11} \lceil \log_2(3/4 * k) \rceil = 0 + 1 + 2 + 2 + 2 + 3 + 3 + 3 + 3 + 3 + 4 = 26$$

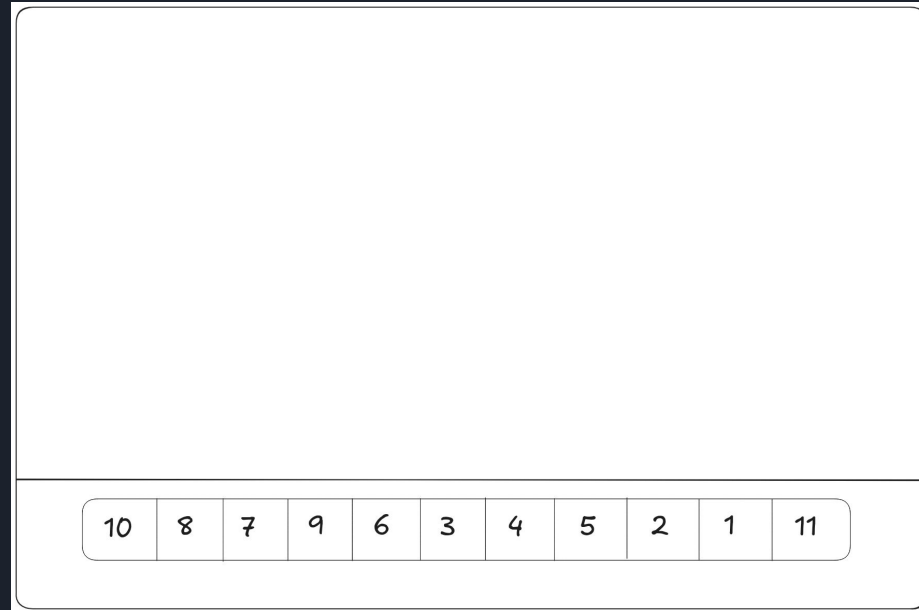
Traduire l'algorithme en code

Merge

étape de
réursion : 0



taille groupe :
 $2^0 = 1$



Merge

étape de
réursion : 0

taille groupe :
 $2^0 = 1$

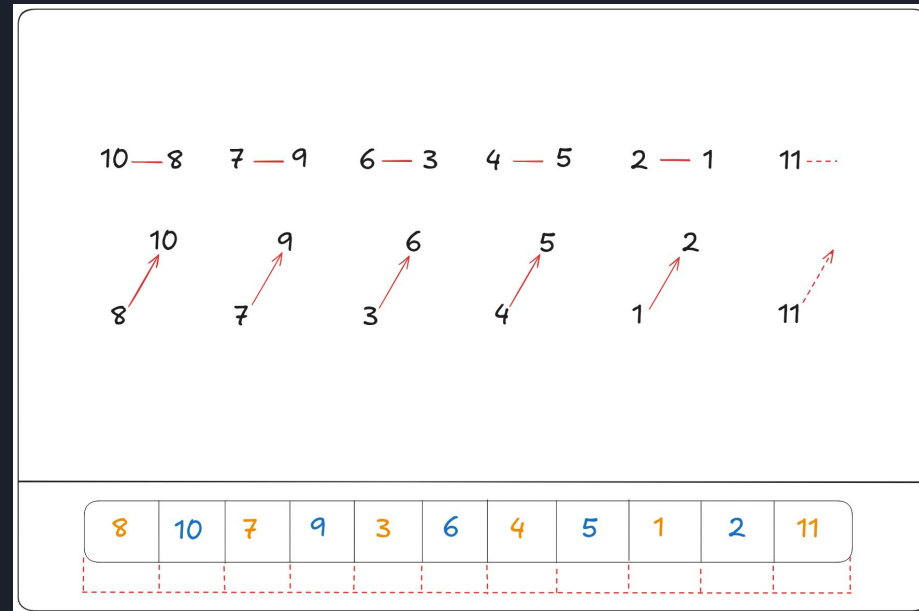
10 — 8 7 — 9 6 — 3 4 — 5 2 — 1 11 ----

10	8	7	9	6	3	4	5	2	1	11
----	---	---	---	---	---	---	---	---	---	----

Merge

étape de
réursion: 0

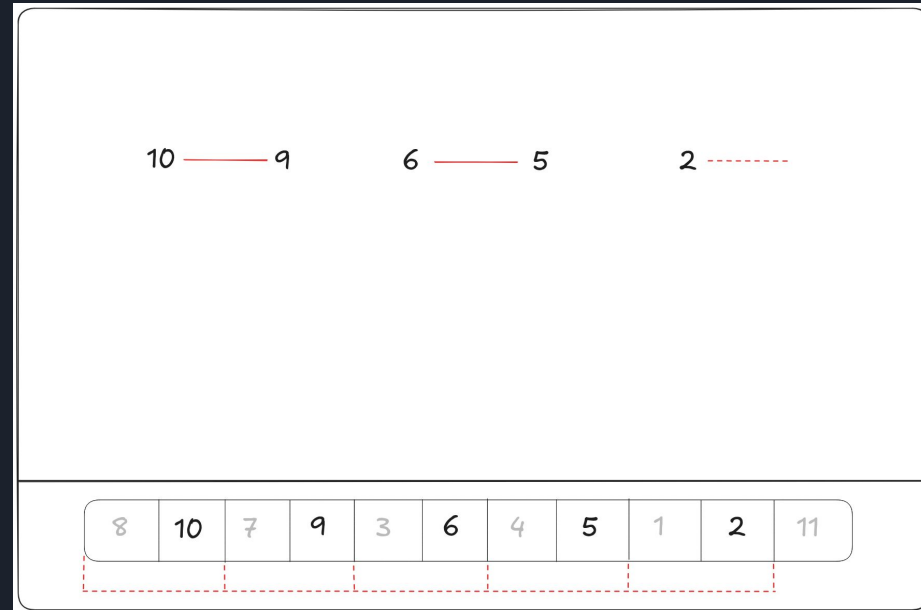
taille groupe:
 $2^0 = 1$



Merge

étape de
réursion : 1

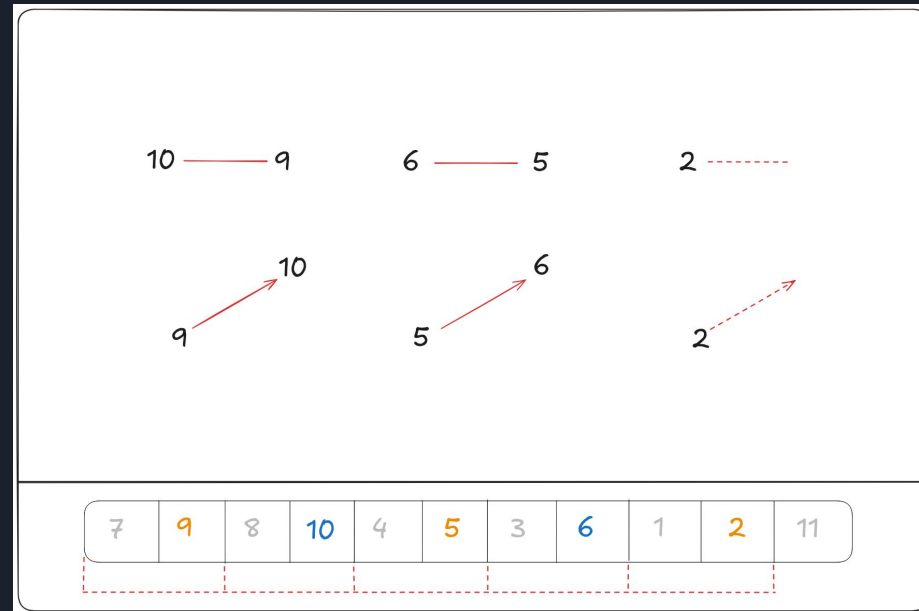
taille groupe :
 $2^1 = 2$



Merge

étape de
réursion : 1

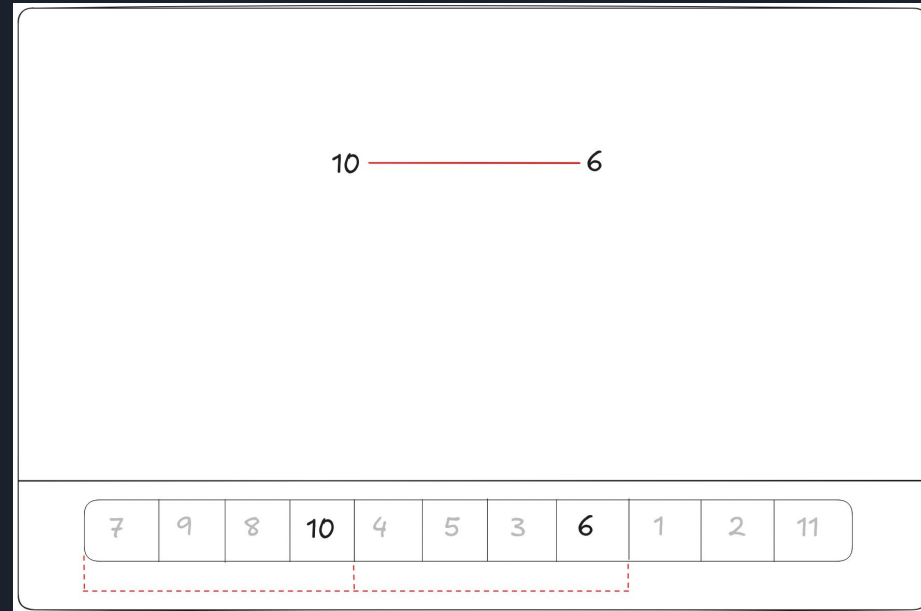
taille groupe :
 $2^1 = 2$



Merge

étape de
réursion : 2

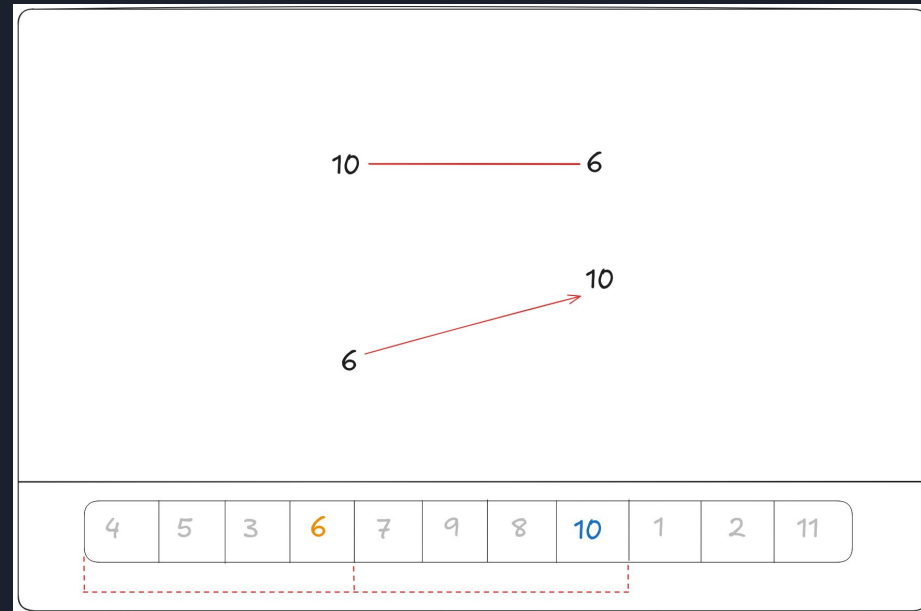
taille groupe :
 $2^2 = 4$



Merge

étape de
réursion : 2

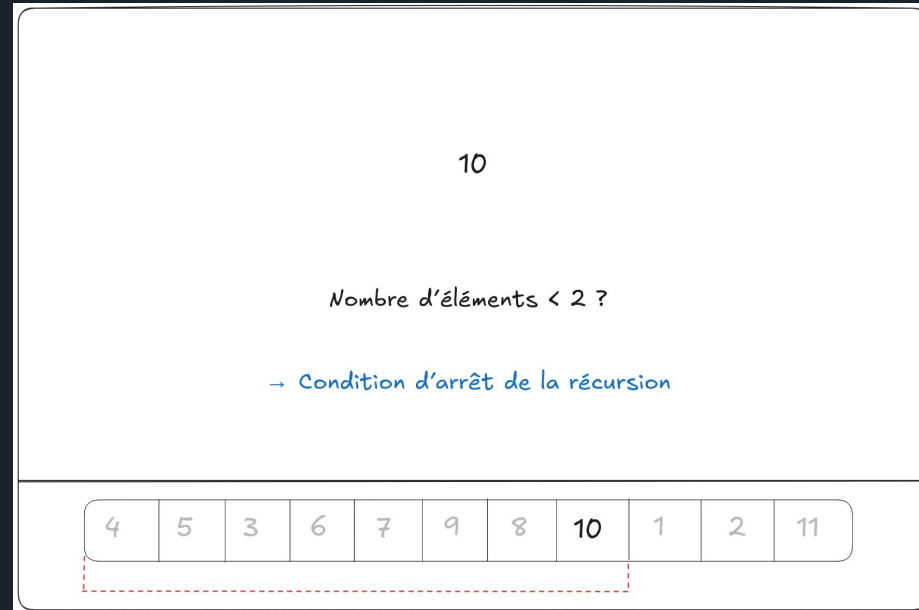
taille groupe :
 $2^2 = 4$



Merge

étape de
réursion : 3

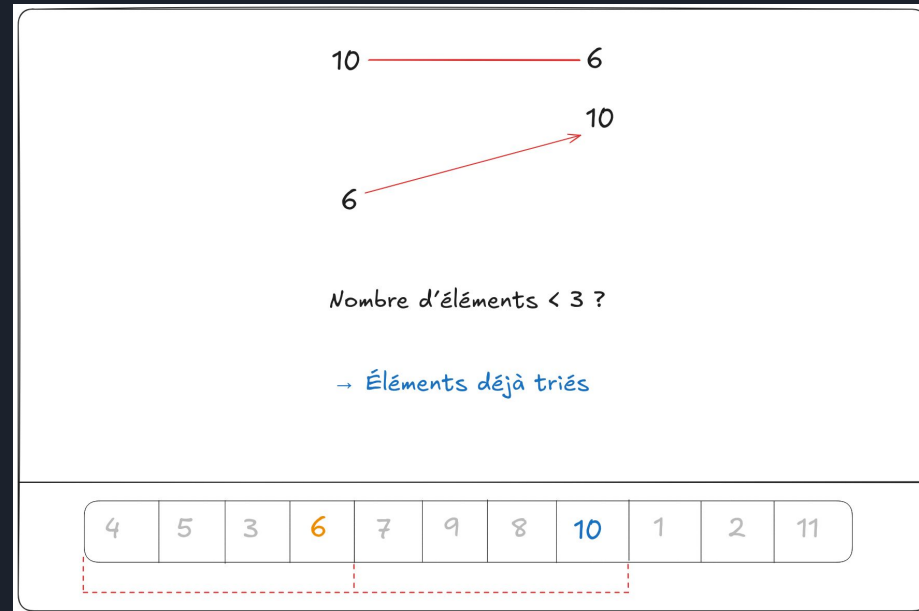
taille groupe :
 $2^3 = 8$



Insertion

étape de
réursion : 2

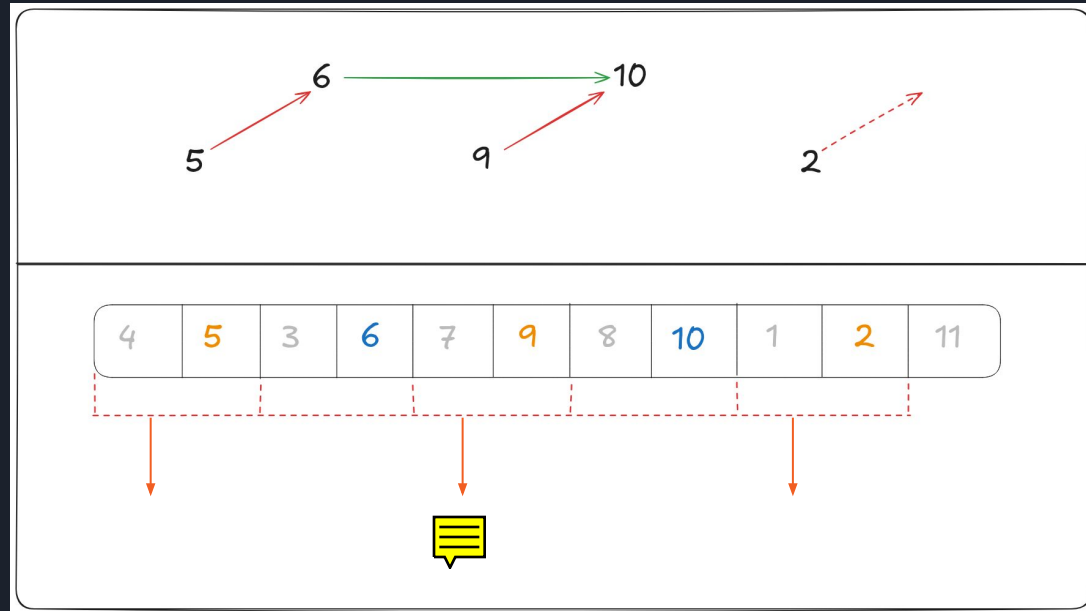
taille groupe :
 $2^2 = 4$



Insertion

étape de
réursion : 1

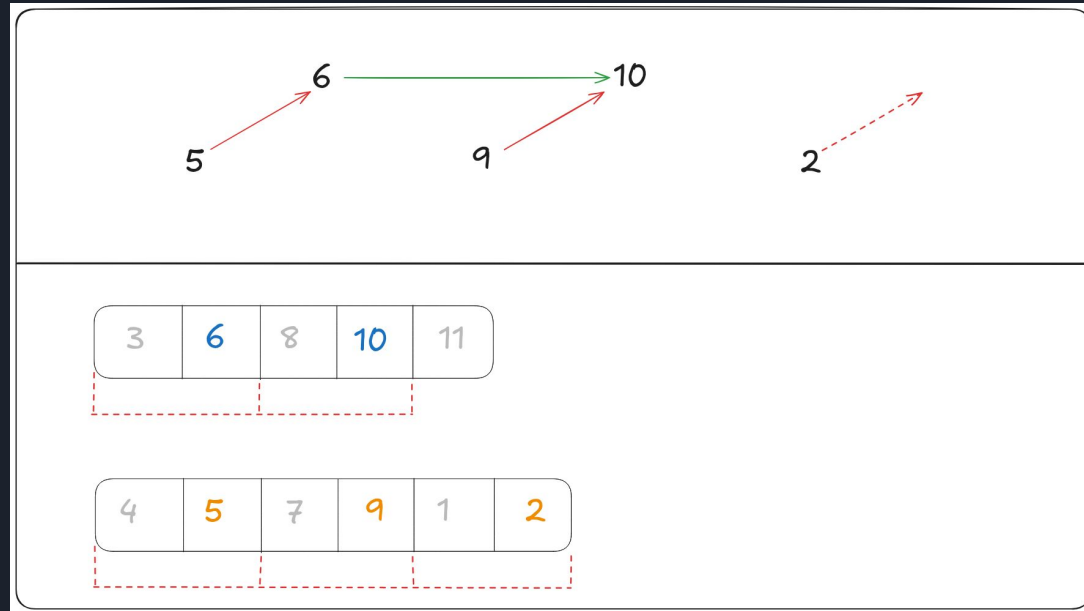
taille groupe :
 $2^1 = 2$



Insertion

étape de
réursion : 1

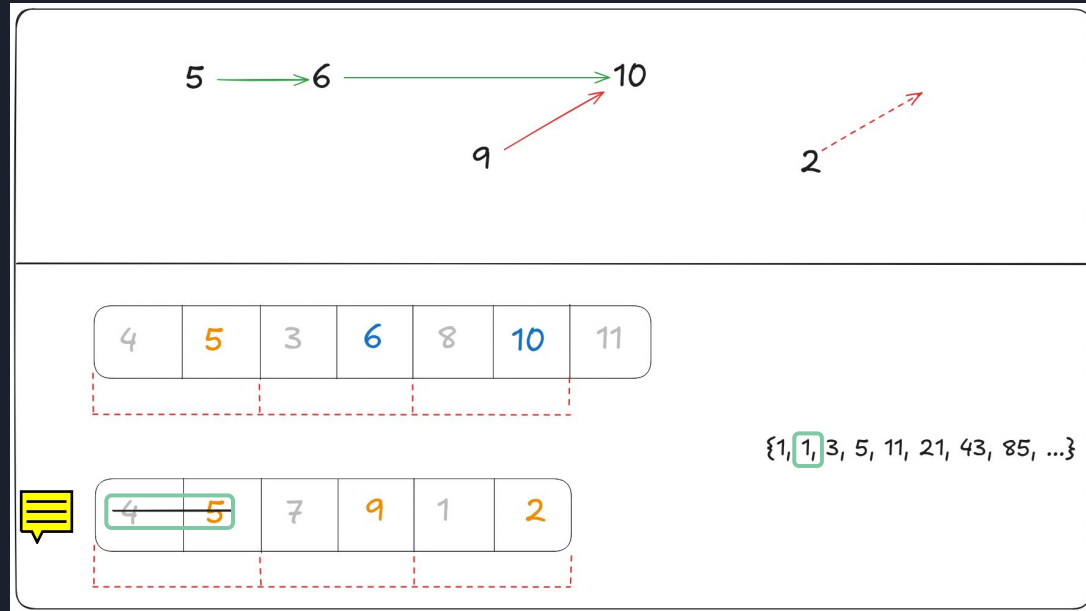
taille groupe :
 $2^1 = 2$



Insertion

étape de
réursion : 1

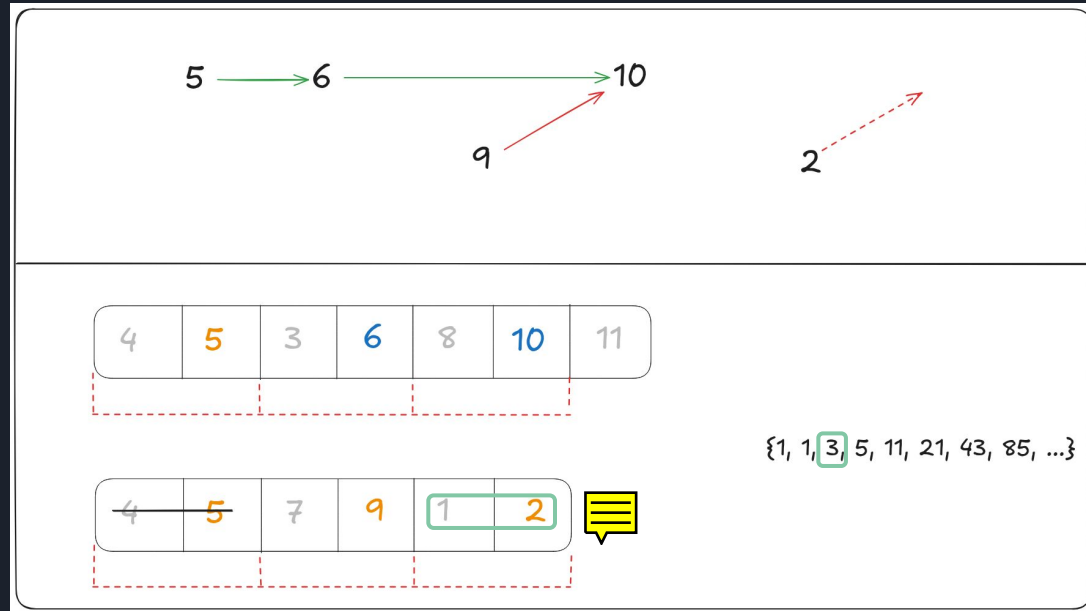
taille groupe :
 $2^1 = 2$



Insertion

étape de
réursion : 1

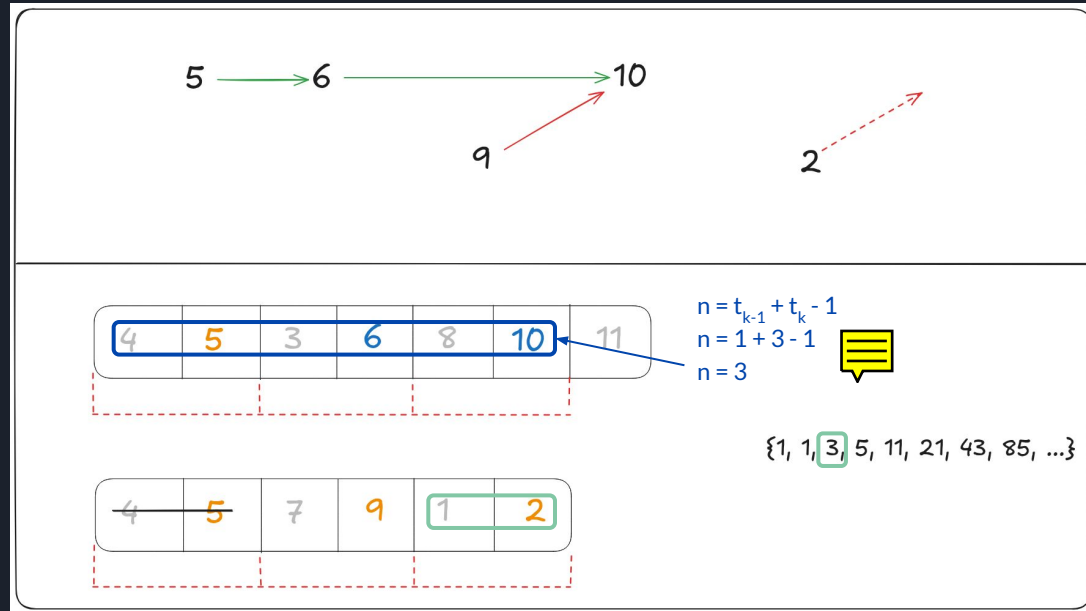
taille groupe :
 $2^1 = 2$



Insertion

étape de
réursion : 1

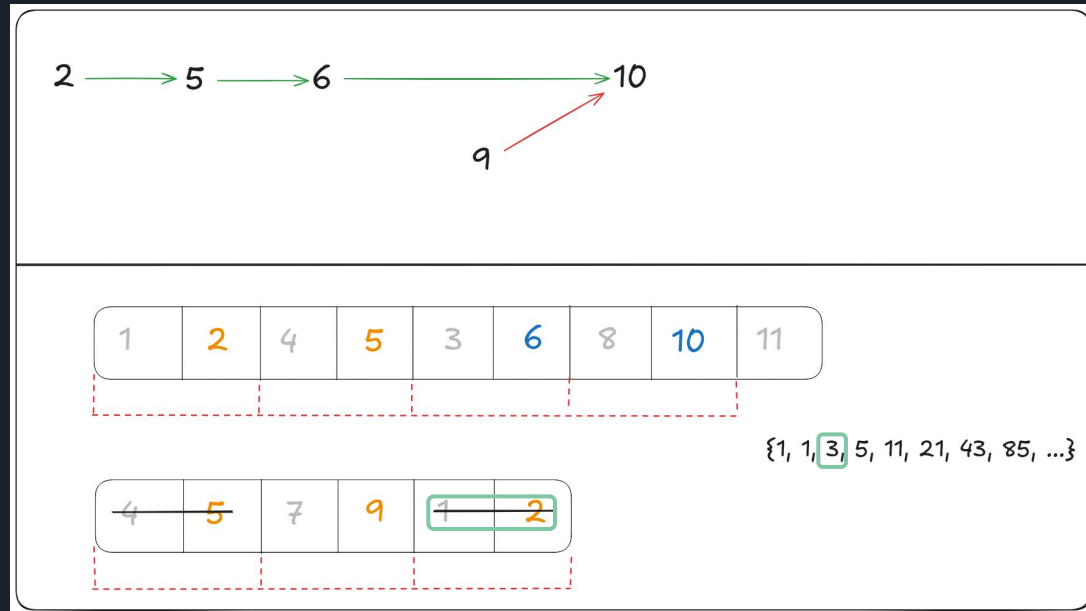
taille groupe :
 $2^1 = 2$



Insertion

étape de
réursion : 1

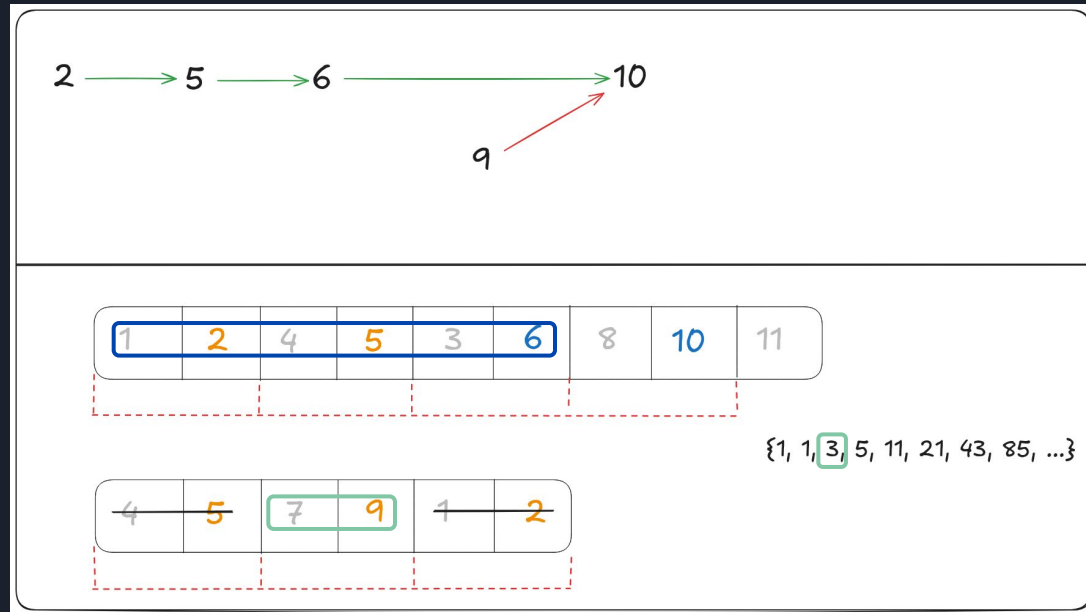
taille groupe :
 $2^1 = 2$



Insertion

étape de
réursion : 1

taille groupe :
 $2^1 = 2$



Insertion

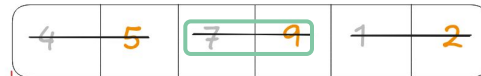
étape de
réursion : 1

taille groupe :
 $2^1 = 2$

2 → 5 → 6 → 9 → 10



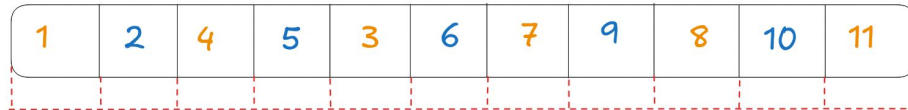
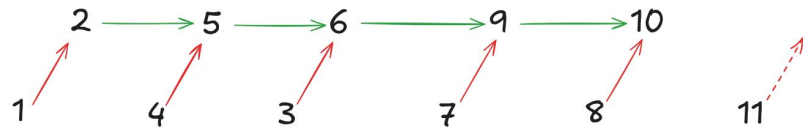
{1, 1, 3, 5, 11, 21, 43, 85, ...}



Insertion

étape de
réursion: 0

taille groupe:
 $2^0 = 1$

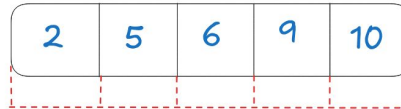
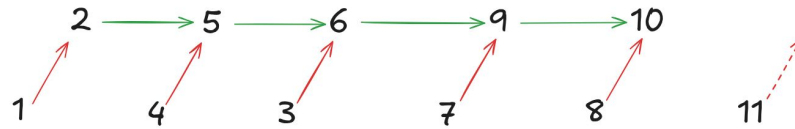


{1, 1, 3, 5, 11, 21, 43, 85, ...}

Insertion

étape de
réursion: 0

taille groupe:
 $2^0 = 1$



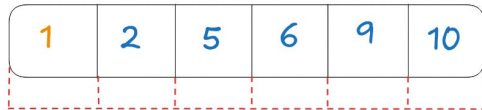
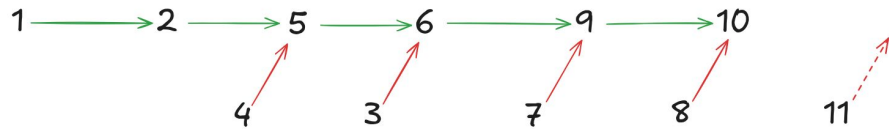
{1, 1, 3, 5, 11, 21, 43, 85, ...}



Insertion

étape de
réursion: 0

taille groupe:
 $2^0 = 1$



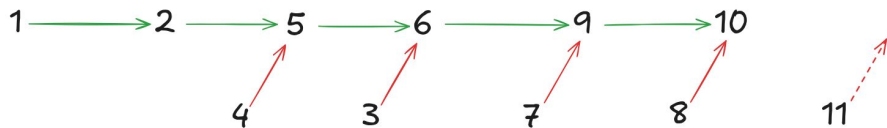
{1, 1, 3, 5, 11, 21, 43, 85, ...}



Insertion

étape de
réursion: 0

taille groupe:
 $2^0 = 1$

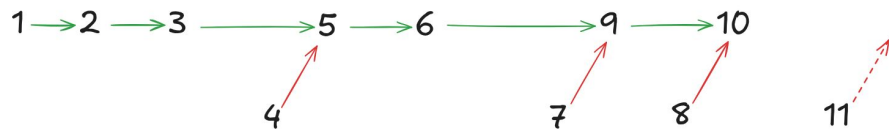


{1, 1, 3, 5, 11, 21, 43, 85, ...}

Insertion

étape de
réursion: 0

taille groupe:
 $2^0 = 1$



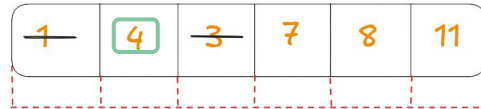
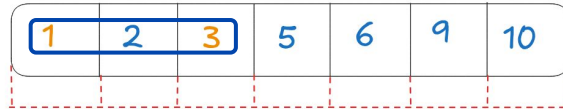
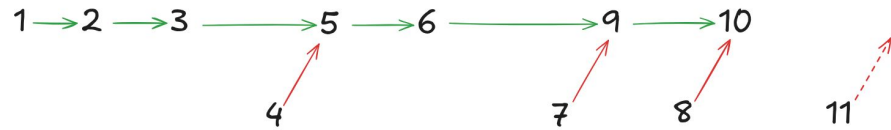
{1, 1, 3, 5, 11, 21, 43, 85, ...}



Insertion

étape de
réursion: 0

taille groupe:
 $2^0 = 1$

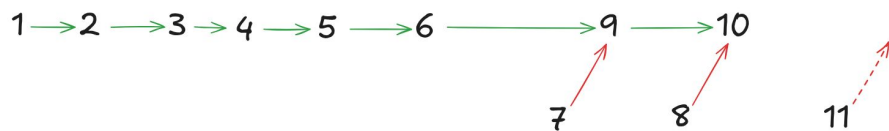


{1, 1, 3, 5, 11, 21, 43, 85, ...}

Insertion

étape de
réursion: 0

taille groupe:
 $2^0 = 1$



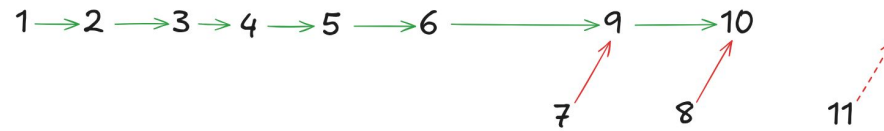
{1, 1, 3, 5, 11, 21, 43, 85, ...}



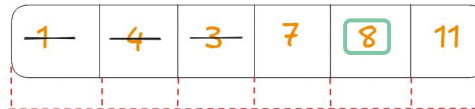
Insertion

étape de
réursion: 0

taille groupe:
 $2^0 = 1$



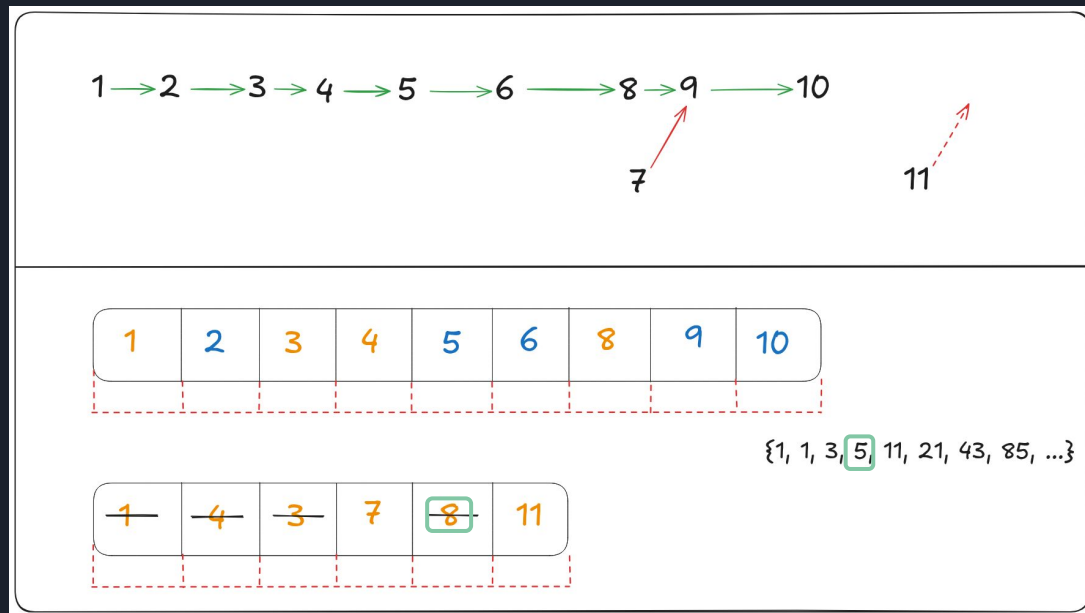
{1, 1, 3, 5, 11, 21, 43, 85, ...}



Insertion

étape de
réursion: 0

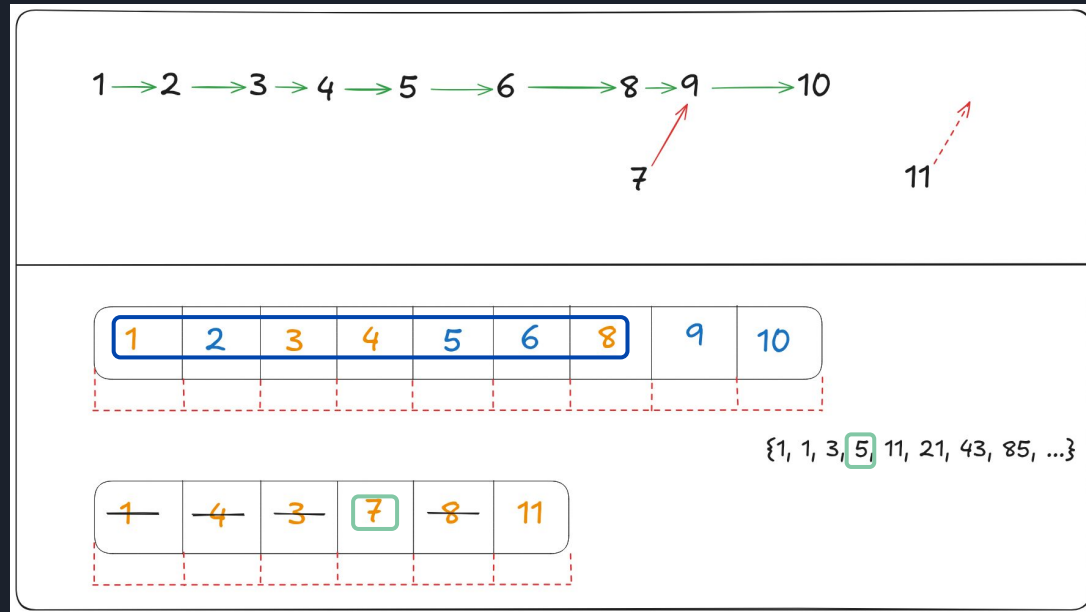
taille groupe:
 $2^0 = 1$



Insertion

étape de
réursion: 0

taille groupe:
 $2^0 = 1$



Insertion

étape de
réursion: 0

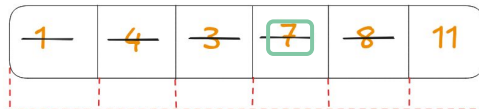
taille groupe:
 $2^0 = 1$

1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10

11



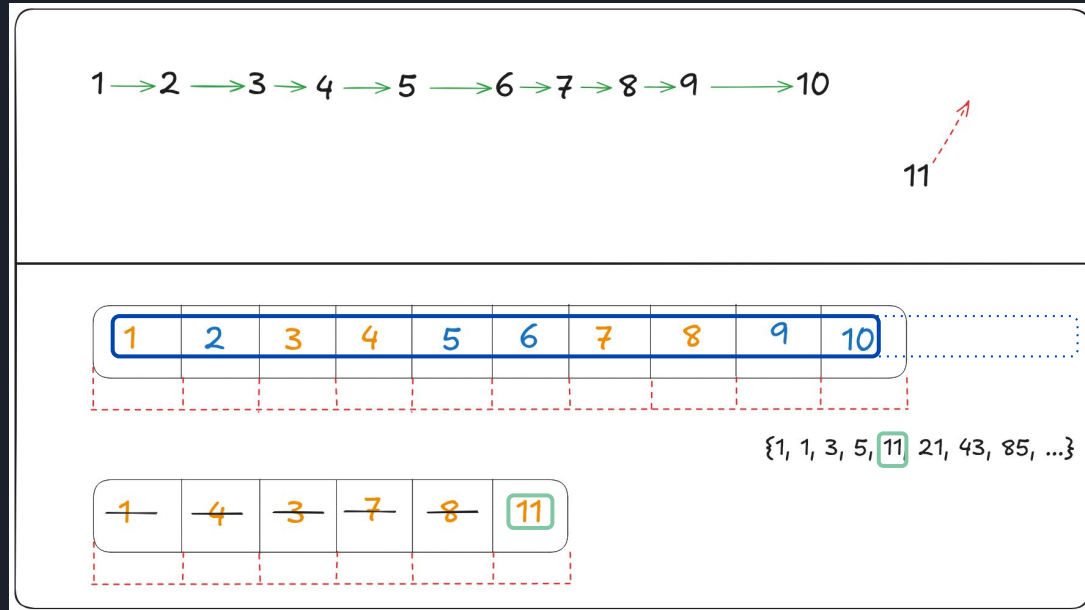
{1, 1, 3, 5, 11, 21, 43, 85, ...}



Insertion

étape de
réursion: 0

taille groupe:
 $2^0 = 1$



Insertion

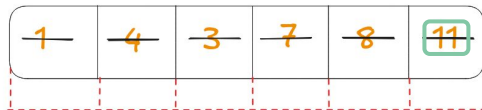
étape de
réursion: 0

taille groupe:
 $2^0 = 1$

1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10 → 11



{1, 1, 3, 5, 11, 21, 43, 85, ...}



Conseils / Rappels



Étapes du projet

- 1) Fonctions d'aide
- 2) Insertion dichotomique
- 3) Merge
- 4) Insertion





Fonctions d'aide

- Vérifie si le tableau est trié
 - paramètre : tableau
 - retour : booléen
- Compte le nombre de groupes du tableau
 - paramètres : taille du tableau, nombre d'éléments par groupe
 - retour : entier positif
- Converti un index de groupe en index de tableau
 - paramètres : nombre d'éléments par groupe, index du groupe
 - retour : entier positif
- Récupère la valeur d'un groupe
 - paramètres : tableau, nombre d'éléments par groupe, index du groupe
 - retour : entier
- Déplace un groupe d'un tableau à un autre (séparation gagnants/perdants)
 - paramètres : tableau, nombre d'éléments par groupe, index du groupe, tableau où déplacer les éléments
- Insère un groupe d'éléments d'un tableau à un autre (insertion des perdants)
 - paramètres : tableau, nombre d'éléments par groupe, index du groupe où insérer, tableau d'où copier les éléments, index du groupe à copier
- Échange 2 groupes d'un tableau (merge)
 - paramètres : tableau, nombre d'élément par groupe, index du premier groupe (, index du deuxième groupe)





Insertion dichotomique

- 1) Recherche dichotomique “simple”
- 2) Insertion dichotomique “simple”
- 3) Insertion dichotomique par groupe d’éléments



Conseils

Containers : vector, puis deque (ou list)



Attributs supplémentaires : nombre de comparaisons du merge, de l'insertion et du total.



Méthodes supplémentaires : nombre de comparaisons maximal théorique du merge, de l'insertion et du total en fonction du nombre d'éléments du tableau.



Rappels

- Interdiction d'invalider un projet si le nombre de comparaisons est supérieur à celui de l'algorithme.
- Le code doit avoir une partie merge et une partie insertion.
- Utilisation de la suite de Jacobsthal obligatoire.
- Vous êtes censé utiliser la recherche/insertion dichotomique.
- Sachez expliquer à quoi servent les formules que vous utilisez...
- ... Mais vous n'avez pas besoin de savoir les démontrer.

Merci de votre attention